



Fact table vs D***ntly killin.mp4
3.8 MB



OLAP & OLTP.mp4
7.0 MB

understand this video and propr audion and that explantion so i
also want to as it is pattern to explain data enginnerr concpet ok so
first -

- 1- give as it script what he cover in video
- 2- that patter guide me 30 topic DE to cover
- 3- i want to generate same content to guide DE concpet .



ChatGPT gpt-5-6

I'll first break down the two videos' actual teaching pattern—hook, explanation flow, examples, visuals, and ending—then turn that into a reusable 30-topic Data Engineering series.

Yes. I understand the pattern you want: **not a normal "teaching lecture"**, but short-form Data Engineering content where the creator speaks directly to the viewer, creates curiosity first, explains one concept through a practical problem, uses simple visual comparisons, and ends with a memorable rule.

I analyzed the structure and visuals of both uploaded videos. The videos are roughly **39–43 seconds**, use a **face-to-camera format**, and insert large text/visual concepts at key moments rather than putting the whole explanation on screen.

1. What the videos are actually doing

Video 1 — Fact Table vs Dimension Table

The core teaching pattern is essentially:

Problem → Consequence → Concept → Simple comparison → Rule

The topic isn't introduced like:

"Today we are going to learn Fact Tables and Dimension Tables."

Instead, it starts with a **problem/attention hook** around putting customer details directly into a fact table.

Then the explanation moves toward:

Fact → Dimension

The visual treatment reinforces the distinction with large labels such as:

FACT | DIMENSION

And near the end, it introduces a memorable decision rule:

“The 2-second decision rule”

That is very important for your own series.

The video is not trying to teach every detail about fact and dimension tables. It gives the viewer **one useful mental model that they can remember immediately.**

Video 2 – OLTP vs OLAP

This follows almost the same philosophy.

Instead of beginning with a textbook definition:

“OLTP stands for Online Transaction Processing...”

the video uses the contrast between the two systems and gradually builds the distinction.

The viewer gets a mental model like:

OLTP → transactions / operational workload

versus

OLAP → analytical workload

The visuals then reinforce the concepts with examples such as cloud/data warehouse technologies.

So again:

Hook → contrast → explanation → real-world example → takeaway

2. The pattern you should use for your DE series

I would NOT make your videos like:

- Definition
- Architecture
- Advantages
- Disadvantages
- Example
- Conclusion

That feels like a classroom lecture.

Instead, use this **short-form DE storytelling framework:**

THE DE CONCEPT FORMULA

0–3 sec — HOOK

Start with something that creates curiosity.

Example:

“Your data pipeline can be perfectly working... and still be wrong.”

3–8 sec — PROBLEM

Show the real engineering problem.

“Because you're loading the same 10 million records every single day.”

8–20 sec — EXPLANATION

Introduce the concept naturally.

“That's exactly why data engineers use incremental loading.”

20–32 sec — VISUAL / EXAMPLE

Show a very simple pipeline.

text

Yesterday
10M records

Today
10,000 new records



Incremental Load



Only 10,000 processed

32–40 sec — ENGINEERING INSIGHT

Give the practical reason.

“You reduce processing time, compute cost, and unnecessary data movement.”

40–45 sec — MEMORABLE RULE

End with something that people can remember.

“If only a small portion of data changes, don't reload everything.”

This is the part similar to the “**2-second decision rule**” from the reference style.

3. Your 30-topic Data Engineering series

I would structure your series into **6 sections**, so it feels like a complete Data Engineering journey rather than 30 random topics.

SECTION 1 — DATA ENGINEERING FUNDAMENTALS

01 — What Does a Data Engineer Actually Do?

Hook:

“Every time you open an app, millions of data points are being generated. But who actually moves all that data?”

Concept:

text

```
graph TD
    Source --> Ingestion
    Ingestion --> Transformation
    Transformation --> Storage
    Storage --> Analytics
```

Ending rule:

“A Data Engineer builds the road that data travels on.”

02 — OLTP vs OLAP

Hook:

“The database handling your Amazon order is not the same system used to analyze millions of orders.”

Explain:

text

OLTP
↓
Transactions

OLAP
↓
Analytics

Ending:

“OLTP runs the business. OLAP analyzes the business.”

03 — Data Lake vs Data Warehouse

Hook:

“Why would a company need both a Data Lake and a Data Warehouse?”

Visual:

text

Data Lake
Raw + Flexible
↓
Data Warehouse
Clean + Structured

Ending:

“Lake stores flexibility. Warehouse serves analytics.”

04 — Data Lakehouse

Hook:

“What if you could get the flexibility of a Data Lake with the structure of a Warehouse?”

Then:

text

Data Lake
+
Data Warehouse
↓
Lakehouse

Introduce:

- Delta Lake
- ACID
- schema
- analytics

05 — Batch vs Streaming

Hook:

“Would you wait until midnight to detect a credit-card fraud?”

Then:

text

Batch

Data → → → → → Process

Streaming

Data → Process → Result

Ending:

“If seconds matter, batch processing may already be too late.”

SECTION 2 — DATA WAREHOUSING

06 — Fact Table vs Dimension Table

Hook:

“If you're putting customer names inside your fact table, you're probably repeating the same information thousands of times.”

Then:

text

FACT

orders

sales

quantity

amount

DIMENSION

customer
product
date

Ending:

“Facts tell you what happened. Dimensions tell you who, what, where, and when.”

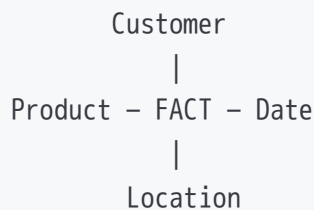
07 — Star Schema

Hook:

“Why do data warehouses often look like a star?”

Visual:

text



Ending:

“One central fact table surrounded by descriptive dimensions.”

08 — Snowflake Schema

Hook:

“What happens when your dimensions become dimensions of dimensions?”

Show:

text



Then explain normalization.

09 — SCD Type 1

Hook:

“A customer changed their city. Should your warehouse remember the old city?”

Then:

text

Before
Pune

After
Mumbai

Type 1:

Old value is overwritten.

10 — SCD Type 2

Hook:

“What if the business wants to know where that customer lived last year?”

Visual:

text

Customer
Pune
2024 → 2025

Customer
Mumbai
2025 → Current

Ending:

“Type 2 doesn't just store the latest truth. It stores the history.”

SECTION 3 — DATA INGESTION & PIPELINES

11 — ETL vs ELT

Hook:

“Should you transform data before storing it, or after storing it?”

text

ETL
Source
↓
Transform
↓
Warehouse

ELT
Source
↓
Warehouse
↓
Transform

Ending:

“Modern cloud platforms made ELT much more practical.”

12 — Full Load vs Incremental Load

Hook:

“Why process 100 million rows when only 5,000 changed?”

Then:

text

Full Load
100M → Process

Incremental
5K → Process

13 — Watermarking

Hook:

“How does your pipeline know which records were already processed?”

Example:

text

Last processed:
2026-10-01 10:00

Next run:
WHERE updated_at > watermark

Ending:

“The watermark tells your pipeline where to continue.”

14 — CDC — Change Data Capture

Hook:

“What if you don't want to repeatedly scan the entire database to find changes?”

Show:

text

```
INSERT  
UPDATE  
DELETE  
↓  
CDC  
↓  
Data Platform
```

15 — Metadata-Driven Pipelines

Hook:

“What if you had 100 tables to ingest? Would you build 100 pipelines?”

Then:

text

```
Metadata  
↓  
Generic Pipeline  
↓  
100 Tables
```

Ending:

“Don't build 100 pipelines when one configurable pipeline can do the job.”

SECTION 4 — CLOUD DATA ENGINEERING

16 — Azure Data Factory

Hook:

“How does a company move data from SQL Server to a cloud data platform automatically?”

Visual:

text

```
graph TD
    A[SQL Server] --> B[ADF Pipeline]
    B --> C[ADLS]
    C --> D[Fabric / Synapse]
```

17 — Linked Service vs Dataset

Hook:

“In ADF, why do you need both a Linked Service and a Dataset?”

Simple analogy:

text

```
Linked Service
= Where / How to connect

Dataset
= What data you're accessing
```

Ending:

“Connection tells you where. Dataset tells you what.”

18 — Integration Runtime

Hook:

“Your ADF pipeline is in Azure. But your database is sitting inside your company's private network. How does ADF reach it?”

Then:

text

ADF
↓
Integration Runtime
↓
Private Network
↓
Database

19 — ADF Pipeline Parameters vs Variables

Hook:

“Why hardcode a table name when the same pipeline can process 100 tables?”

Explain:

text

Parameter
= Input

Variable
= Runtime value

20 — Dynamic Content

Hook:

“How does one pipeline automatically pick today's folder?”

Example:

text

/raw/2026/10/02/

using dynamic expressions.

Ending:

“Dynamic pipelines replace hardcoded values with runtime logic.”

SECTION 5 — BIG DATA & SPARK

21 — Why Do We Need Spark?

Hook:

“What happens when your dataset becomes too large for one machine?”

Then:

text

1 Machine



Too much data

Spark



Machine 1

Machine 2

Machine 3

Machine 4

Ending:

“Spark distributes the work instead of forcing one machine to do everything.”

22 — Spark DataFrame

Hook:

“Why do Data Engineers prefer DataFrames instead of manually processing every row?”

Explain:

text

Raw Data



DataFrame



Filter

Transform

Aggregate

23 — Partitioning

Hook:

“Your Spark job is slow even though you have a cluster. Why?”

Then explain:

text

```
Data
↓
Partitions
↓
Parallel Processing
```

Then introduce the danger of:

too few partitions / too many partitions

24 – Shuffle

Hook:

“This one Spark operation can suddenly make your job much slower.”

Then:

text

```
Partition 1 ─┐
Partition 2 ─┴─→ SHUFFLE → New Partitions
Partition 3 ─┐
Partition 4 ─┐
```

Explain why joins/groupBy can trigger shuffle.

25 – Broadcast Join

Hook:

“What if one table is tiny and the other table has billions of rows?”

Then:

text

```
Large Table
↓
JOIN
↑
Small Table → Broadcast
```

Ending:

“Don't move the giant table when you can copy the small one.”

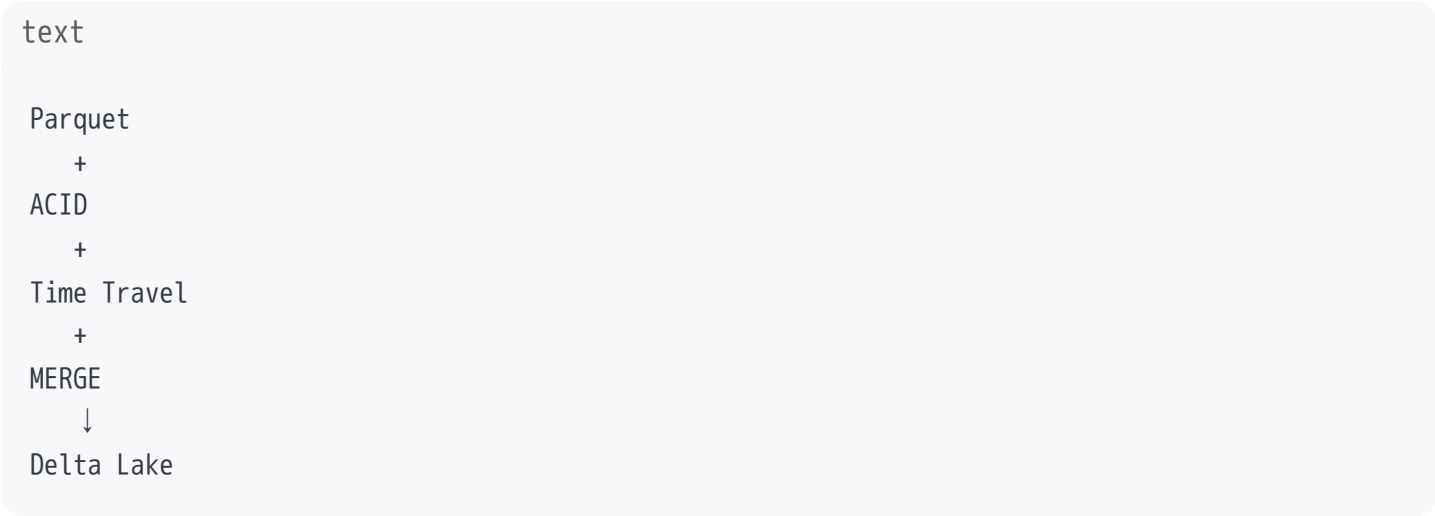
SECTION 6 – MODERN DATA ENGINEERING

26 – Delta Lake

Hook:

“Parquet stores data. But what happens when you need UPDATE, DELETE, or reliable transactions?”

Then:

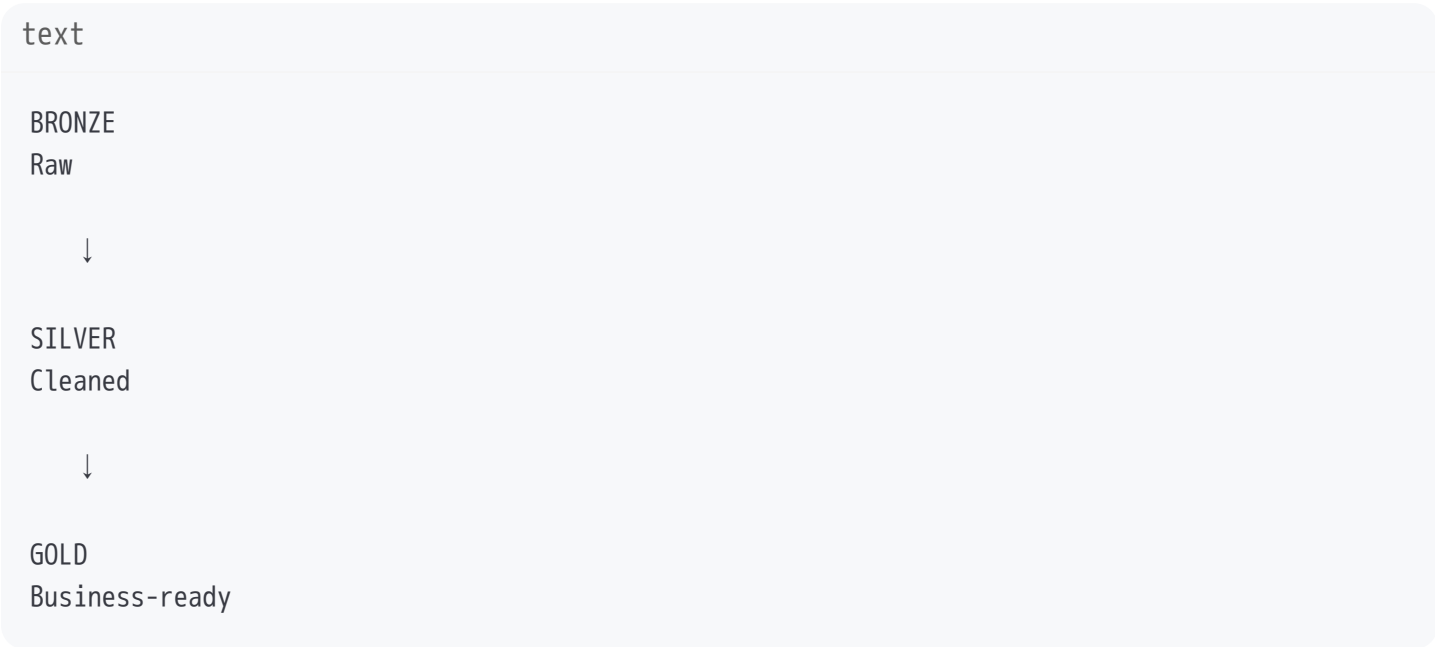


27 – Medallion Architecture

Hook:

“Why do modern data platforms separate data into Bronze, Silver and Gold?”

Visual:



Ending:

“Bronze stores it. Silver fixes it. Gold serves it.”

28 — Data Quality

Hook:

“A pipeline can succeed technically and still deliver completely wrong data.”

Then show:

text

Pipeline SUCCESS

≠

Data SUCCESS

Cover:

- null checks
- duplicates
- schema validation
- freshness
- referential integrity

29 — Data Orchestration & Monitoring

Hook:

“Your pipeline ran successfully at 2 AM. How do you know?”

Show:

text

Pipeline

↓

Logs

↓

Metrics

↓

Alerts

↓

Engineer

Ending:

“A production pipeline isn't complete until you can detect when it fails.”

30 — End-to-End Data Engineering Architecture

This becomes your **final episode**.

Hook:

“Now let's connect everything you've learned into one real Data Engineering system.”

Architecture:

text



Then bring in:

Incremental Load

CDC

SCD

Spark

Delta

Data Quality

Monitoring

This gives the viewer the feeling:

“Okay, now I understand how all these individual concepts fit together.”

4. But the MOST important part: your script pattern

For every one of those 30 videos, I recommend that we **do not simply write a definition**.

We will use this exact structure:

[HOOK]

One sentence that creates curiosity.

[PROBLEM]

Show the real-world problem.

[CONCEPT]

Introduce the DE concept naturally.

[VISUAL]

A simple diagram/animation that matches the spoken sentence.

[REAL-WORLD EXAMPLE]

Amazon / Netflix / banking / e-commerce / Microsoft-style example where appropriate.

[ENGINEERING INSIGHT]

Tell the viewer **why a Data Engineer actually cares**.

[MEMORY RULE]

One short sentence they remember.

Example — how I would write your actual video

Topic: Incremental Loading

0–3 sec — HOOK

“Imagine you have 100 million records in your database... but today only 10,000 records changed.”

Visual:

100M records appears on screen.

3–8 sec — PROBLEM

“Would you really process all 100 million records every single day?”

Visual:

100M → ❌

8–15 sec — CONCEPT

“That's exactly where incremental loading comes in.”

Visual:

text

100M records



Find only changed records



10K records

15–27 sec — EXPLANATION

“Instead of loading everything again, we identify the records that were inserted or updated after our last successful pipeline run.”

Visual:

text

Last Run

10:00 AM



Watermark



New / Updated Records

27–37 sec — ENGINEERING VALUE

“That means less data movement, less compute, faster pipelines and lower cost.”

Show each phrase appearing with the voice.

37–43 sec — MEMORY RULE

“If only a small part of your data changes, don't reload everything.”

That final line is the equivalent of the reference video's **memorable decision rule**.

5. The visual style I recommend for YOUR videos

Based on the reference videos, keep the visual language consistent across all 30.

Face

You can use your face as the main speaker.

But don't keep your face full-screen for the entire video.

Use:

Face → concept visual → face → diagram → face

Text

Don't put paragraphs on screen.

Use **2–5 words**.

For example:



Incremental loading is a technique used to load only newly created or modified data...

Instead:

FULL LOAD ❌

INCREMENTAL LOAD ✅

ONLY CHANGED DATA

Diagrams

Keep them extremely simple.

For example:

text

```
graph TD
    SOURCE --> ADF
    ADF --> BRONZE
    BRONZE --> SILVER
    SILVER --> GOLD
```

Not a giant enterprise architecture diagram.

Voice

Your voice should sound like you're explaining to **one person**, not presenting a PPT.

For example:

“Okay, but here's the interesting part...”

“Now think about this...”

“This is where most beginners get confused...”

“And this is exactly why we use...”

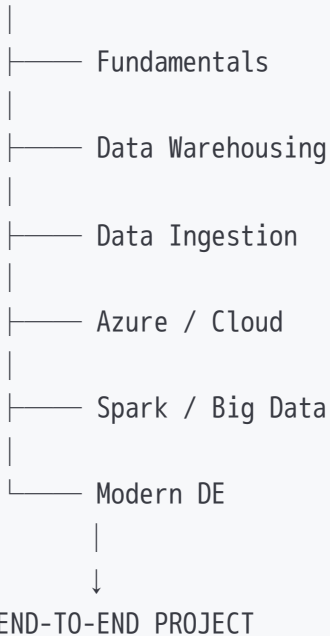
These small transitions make the AI voice sound much more human too.

6. Your 30-video journey

The overall progression becomes:

text

DATA ENGINEERING



And importantly, **the topics build on each other**.

For example:

Fact vs Dimension → Star Schema → SCD → Warehouse

then:

Full Load → Incremental → Watermark → CDC

then:

Spark → Partition → Shuffle → Broadcast

then:

Bronze → Silver → Gold → Delta → Data Quality

So after 30 videos, someone doesn't just know 30 definitions—they have a **mental model of how a real Data Engineering platform works**.

One important adjustment

The reference videos are around **40 seconds**, so I would target **35–50 seconds per DE video** rather than trying to squeeze too much into 60–90 seconds.

For deeper concepts like **SCD Type 2, CDC, Spark Shuffle, Delta Lake**, we can use a **Part 1 / Part 2** approach rather than overcrowding one reel.

If you want to proceed with this exact style, **the next step should be Day 1: “What Does a Data Engineer Actually Do?”** and I can write the **complete recording-ready Hindi + English script + exact voice pauses + on-screen text + diagram/animation instructions + timestamp-by-timestamp editing plan**, matching this reference pattern.

2026-10-02 10:09:26 You 

your not mention proper contnet from stepwise and all topic required and check connection concepts

 ChatGPT gpt-5-6

You're right—the previous list was a topic list, not a connected learning path. I'll rebuild it as a **dependency-based DE curriculum**, where each video teaches the concepts needed for the next one and the reference-video pattern is applied consistently.

Yes — your point is correct. You don't want **30 independent DE topics**. You want a **connected 30-video curriculum**, where:

Video 1 creates the knowledge required for Video 2 → Video 2 supports Video 3 → ... → Video 30 completes a real Data Engineering system.

I also re-checked the uploaded videos frame-by-frame. The important thing I missed earlier is that the reference creator is not simply “explaining a topic.” He uses a **very specific teaching mechanism**.

1. The actual reference pattern

Video A — Fact Table vs Dimension Table

The flow is approximately:

Problem → show bad design → introduce two concepts → compare → give decision rule → leave a question

The important visual sequence is:

text

Customer details repeated
↓
Why is this a problem?
↓
FACT TABLE DIMENSION TABLE
Numeric data Lookup context
Many rows Descriptive data
↓
Customer Age:
Fact or Dimension?
↓
2-second decision rule

The ending visual gives the actual usable rule:

text

SUM / AVG ?
↓
FACT TABLE

FILTER / GROUP BY ?
↓
DIMENSION TABLE

That is the **real strength** of the reference.

It doesn't end with:

“So that's what a fact table is.”

It ends with a **decision framework**.

Video B — OLTP vs OLAP

The second video follows the same idea:

text

OLTP vs OLAP
↓
Different workloads
↓
OLAP
Millions of rows
Aggregation
↓
Technology examples
↓
MongoDB:
OLTP or OLAP?
↓
Application
↓
OLTP
↓
Data Pipeline
↓
OLAP Warehouse
↓
Power BI

So the creator isn't just teaching:

OLTP = X
OLAP = Y

He is teaching:

When would I use X instead of Y, and where does each one fit in a real system?

That's the pattern we should copy for your DE content.

2. The correct 30-topic DE learning path

I'm changing the earlier list significantly.

We need to start with **how data exists**, then **how businesses store it**, then **how we model it**, then **how we move it**, then **how we process it**, then **how we build production platforms**.

Here is the dependency chain:

text

DATA
↓

DATABASES
↓
OLTP / OLAP
↓
DATA WAREHOUSE
↓
DATA LAKE
↓
LAKEHOUSE
↓
DATA MODELING
↓
FACT / DIMENSION
↓
STAR SCHEMA
↓
SCD
↓
ETL / ELT
↓
BATCH / STREAMING
↓
FULL / INCREMENTAL
↓
WATERMARK / CDC
↓
ADF / ORCHESTRATION
↓
SPARK
↓
PARTITIONING
↓
SHUFFLE / JOINS
↓
DELTA LAKE
↓
MEDALLION
↓
DATA QUALITY
↓
SECURITY / GOVERNANCE
↓
MONITORING
↓
END-TO-END ARCHITECTURE

Now let's make this into **30 actual videos**.

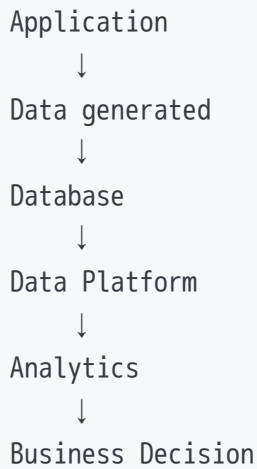
PHASE 1 – UNDERSTAND THE DATA WORLD

01 – What Actually Happens to Data?

Prerequisite: None

Teach:

text



The viewer must understand the **journey of data** before learning individual technologies.

Connection

This naturally leads to:

“Okay, but where does the application actually store this data?”

→ Video 2.

02 – Database vs Data Warehouse vs Data Lake

Teach the three major storage purposes.

text



Don't introduce Star Schema yet.

Connection

Now the viewer knows there are different systems.

Next question:

“Why can't my application database simply be my analytics database?”

→ Video 3.

03 – OLTP vs OLAP

This is where the reference video belongs.

text

OLTP
Application
Transactions
INSERT / UPDATE
Small operations

OLAP
Analytics
Aggregations
Large scans

Then:

text

APP
↓
OLTP
↓
Data Pipeline
↓
OLAP
↓
BI

Decision rule

text

Running the business?

→ OLTP

Analyzing the business?

→ OLAP

Connection

Now:

“If OLAP is for analytics, how should we organize the data inside it?”

→ Data Warehouse.

PHASE 2 — DATA WAREHOUSE & MODELING

04 — What Is a Data Warehouse?

Teach:

- centralized analytical storage
- historical data
- structured data
- reporting
- BI

Example:

text

Sales DB

CRM

Payments

Website

↓

Data Warehouse

↓

Power BI

Connection

Now ask:

“But if I have billions of rows, how do I design the warehouse?”

→ Data modeling.

05 — Data Modeling: Why Does It Matter?

Teach:

text

Business



Entities



Relationships



Tables



Analytics

Example:

text

Customer

Product

Order

Date

Store

Introduce:

Fact vs Dimension

but don't deeply explain them yet.

Connection

→ Video 6.

06 — Fact Table vs Dimension Table

Now use the **reference-video pattern almost exactly**.

Start with:

“Putting customer details inside your fact table can silently create a huge problem.”

Then:

text

FACT
Sales
Quantity
Revenue
Order ID

DIMENSION
Customer
Product
Date
Store

Decision rule:

text

SUM / AVG?
→ FACT

FILTER / GROUP BY?
→ DIMENSION

Connection

Now the viewer understands the two table types.

Next question:

“How do these tables connect?”

→ Star Schema.

07 – Star Schema

Teach:

text

```
graph TD
    DimCustomer[Dim Customer] --- FactSales[Fact Sales]
    DimProduct[Dim Product] --- FactSales
    DimDate[Dim Date] --- FactSales
    DimStore[Dim Store] --- FactSales
```

Explain:

- central fact
- surrounding dimensions

- foreign keys
- analytical queries

Connection

Then ask:

“Why not split those dimensions into more tables?”

→ Snowflake Schema.

08 — Star Schema vs Snowflake Schema

Teach:

text

STAR

Fact



versus:

text

SNOWFLAKE

Fact



Product



Category



Department

Explain:

Denormalization vs normalization

Connection

Now we have the structure.

Next issue:

“What happens when customer information changes?”

→ SCD.

09 – Slowly Changing Dimensions

First explain the problem.

text

Customer
Pune

Later:

text

Customer
Mumbai

Question:

“Do we overwrite Pune?”

Then introduce:

- Type 1
- Type 2

Connection

10 – SCD Type 1 vs Type 2

Make this a dedicated practical video.

Type 1

text

Pune
↓
Mumbai

Old value disappears.

Type 2

text

Customer	City	Start	End
101	Pune	2024	2025
101	Mumbai	2025	NULL

Decision:

text

Need history?

→ Type 2

Don't need history?

→ Type 1

Now the warehouse modeling foundation is complete.

PHASE 3 – HOW DATA GETS INTO THE PLATFORM

11 – ETL vs ELT

Now ask:

“We designed our warehouse. But how does data get there?”

text

ETL

Source

↓

Transform

↓

Load

versus:

text

ELT

Source

↓

Load

↓

Transform

Explain why cloud platforms made ELT popular.

Connection

But not all data arrives the same way.

→ Batch vs Streaming.

12 — Batch vs Streaming

text

BATCH

Data



Wait



Process

versus:

text

STREAMING

Event



Process



Result



Event



Process

Examples:

Batch: daily sales report

Streaming: fraud detection

Connection

Now ask:

“How do we actually move batch data?”

→ ingestion.

13 — Data Ingestion

Teach source categories:

text

SQL
API
CSV
JSON
Files
SaaS
Events

Then:

text

Sources
↓
Ingestion Layer
↓
Data Platform

Explain:

- source
- extraction
- landing
- ingestion

Connection

Now introduce the Azure implementation.

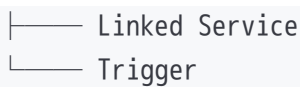
→ ADF.

14 — Azure Data Factory: Pipeline Fundamentals

Teach:

text

Pipeline
|—— Activity
|—— Dataset



Don't overload the viewer with every ADF feature.

Explain the pipeline as:

A workflow that coordinates data movement and processing.

Connection

Now the viewer needs to understand the pieces.

15 – Linked Service vs Dataset vs Pipeline

This should be a very practical comparison.

text

Linked Service
= CONNECTION

Dataset
= DATA STRUCTURE / LOCATION

Pipeline
= WORKFLOW

Example:

text

```
ADF
↓
Linked Service → SQL Server
↓
Dataset → Orders
↓
Copy Activity
↓
ADLS
```

Connection

Next problem:

“Should we copy the entire Orders table every day?”

→ Full vs Incremental.

PHASE 4 — REAL PIPELINE ENGINEERING

16 — Full Load vs Incremental Load

Example:

text

Orders = 100M rows

Today's changes = 10K

Full load:

text

100M → Process

Incremental:

text

10K → Process

Decision:

“If only a small portion changes, don't reload everything.”

Connection

But now:

“How does the pipeline know which 10K rows changed?”

→ Watermark.

17 — Watermark-Based Incremental Loading

Teach:

text

Last successful timestamp

↓

10:00 AM

Next run:

```
WHERE updated_at > 10:00 AM
```

Then:

text

Source



Filter changed rows



Target



Update watermark

Very important:

Update the watermark only after successful processing.

Connection

But timestamp-based logic isn't always enough.

→ CDC.

18 – CDC: Change Data Capture

Teach:

text

INSERT

UPDATE

DELETE



CDC



Target

Compare:

text

Watermark

→ Detect based on column/value

CDC

→ Capture actual changes

Connection

Now the viewer understands incremental ingestion.

Next question:

“What if I have 500 tables?”

→ Metadata-driven pipelines.

19 — Metadata-Driven Pipelines

Instead of:

```
text
```

```
Table 1 → Pipeline 1  
Table 2 → Pipeline 2  
Table 3 → Pipeline 3  
...  
Table 500 → Pipeline 500
```

Build:

```
text
```

```
Metadata Table  
  ↓  
Generic Pipeline  
  ↓  
ForEach  
  ↓  
500 Tables
```

Teach:

- configuration
- parameterization
- reusable pipelines
- control tables

Connection

Now:

“We can move the data. But where should raw, cleaned and business-ready data live?”

→ Lakehouse architecture.

20 — Data Lake vs Lakehouse

Now the viewer already understands warehouse + ingestion.

Teach:

Data Lake

text

Cheap
Flexible
Raw
Files

Lakehouse

text

Data Lake
+
Warehouse capabilities
↓
Lakehouse

Introduce:

- Delta
- ACID
- schema
- transactions
- analytics

Don't go deep into Delta yet.

PHASE 5 — LAKEHOUSE + BIG DATA

21 — Medallion Architecture

Now introduce:

text

SOURCE
↓

BRONZE
Raw
↓
SILVER
Cleaned
↓
GOLD
Business-ready

Decision rule:

Bronze = preserve raw data
Silver = clean and standardize
Gold = serve business use cases

Connection

But processing billions of records requires distributed compute.

→ Spark.

22 — Apache Spark: Why Do We Need It?

Start with:

“What happens when 1 TB of data doesn't fit comfortably into one machine?”

Then:

text

1 Machine



Spark Cluster

Machine 1

Machine 2

Machine 3

Machine 4

Explain:

distributed processing

Connection

Now:

“How does Spark divide that data?”

→ partitions.

23 – Spark Partitions

Teach:

```
text

Large Dataset
  ↓
Partition 1
Partition 2
Partition 3
Partition 4
  ↓
Parallel Processing
```

Explain:

- partition
- parallelism
- task

Then introduce the problem:

“But what happens when data has to move between partitions?”

→ Shuffle.

24 – Spark Shuffle

This is an important DE concept that shouldn't be skipped.

Example:

```
text

GROUP BY
JOIN
ORDER BY
```

can cause:

```
text

Partition 1 ┌
Partition 2 ┤ ────> SHUFFLE
```

Partition 3 —|
Partition 4 —|

Explain:

data movement = expensive

Connection

Then:

“Can we avoid unnecessary movement?”

→ joins / broadcast.

25 — Spark Joins & Broadcast Join

Explain normal join:

text

Large Table
+
Large Table
↓
Shuffle

Then:

text

Large Table
+
Small Table
↓
Broadcast

Decision:

Small dimension + huge fact → broadcast may be useful.

This connects perfectly with your earlier **fact/dimension** lesson.

That's an example of a concept connection we should deliberately create throughout the series.

PHASE 6 — MODERN PRODUCTION DATA ENGINEERING

26 — Delta Lake & MERGE

Now the viewer knows:

- lakehouse
- Spark
- incremental data
- SCD

So Delta finally makes sense.

Teach:

text

Parquet

+

ACID

+

UPDATE

+

DELETE

+

MERGE

+

Time Travel

↓

Delta Lake

Then show:

sql

MERGE INTO target

USING source

Conceptually:

text

New record → INSERT

Existing record → UPDATE

This connects directly to **incremental loading + SCD Type 2**.

27 — Data Quality

Now ask:

“What if the pipeline succeeds but the data is wrong?”

Important distinction:

```
text

Pipeline SUCCESS
    ≠
Data SUCCESS
```

Cover:

- null checks
- duplicate checks
- schema validation
- row count
- referential integrity
- freshness

Example:

```
text

Orders
Customer_ID
    ↓
Does customer exist?
```

This connects back to fact/dimension modeling.

28 — Data Security & Governance

Now the platform is becoming production-ready.

Teach:

```
text

Who can access?
What can they access?
Where is data stored?
Who changed it?
```

Concepts:

- RBAC

- permissions
- sensitive data
- lineage
- catalog
- governance

Don't turn this into a security certification lesson.

The goal is:

A production data platform needs controlled and traceable data access.

29 — Pipeline Monitoring & Failure Handling

Now ask:

“Your pipeline runs at 2 AM. How will you know it failed?”

Architecture:

text

```
graph TD
    Pipeline --> Logs
    Logs --> Metrics
    Metrics --> Alerts
    Alerts --> Engineer
```

Teach:

- retry
- failure path
- logging
- alerts
- monitoring
- idempotency

This is where the series moves from:

“I know DE concepts”

to:

“I understand production DE.”

30 – Build the Complete Data Engineering Architecture

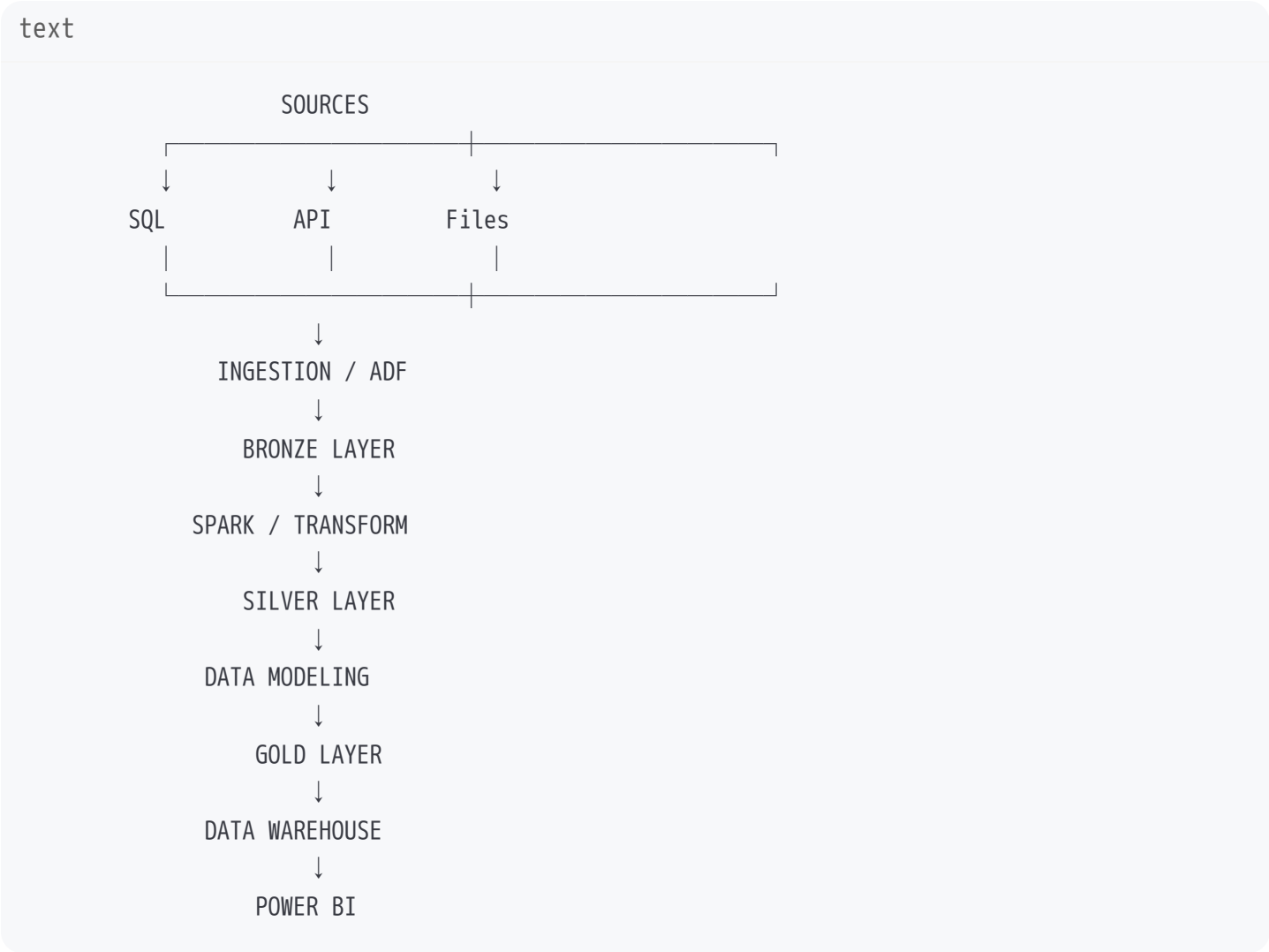
This should NOT be another definition video.

It should be the **payoff of all 29 previous videos**.

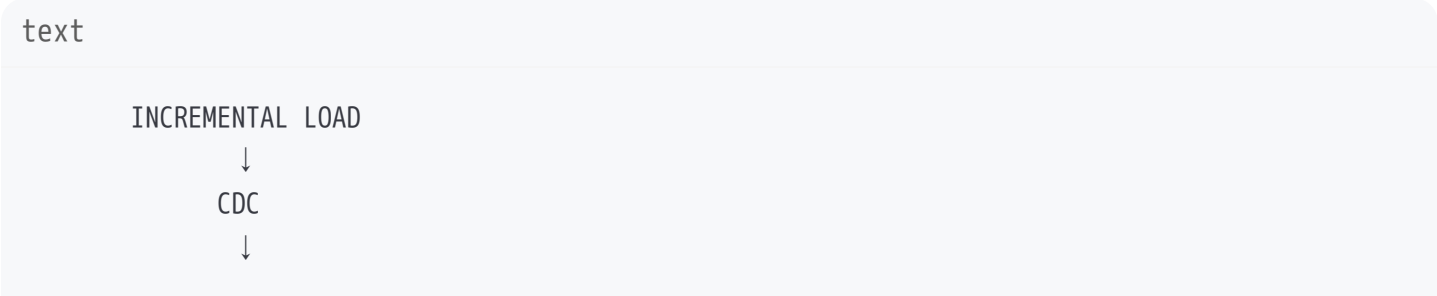
Start:

“You’ve learned 29 Data Engineering concepts. Now let’s connect them into one real pipeline.”

Then:

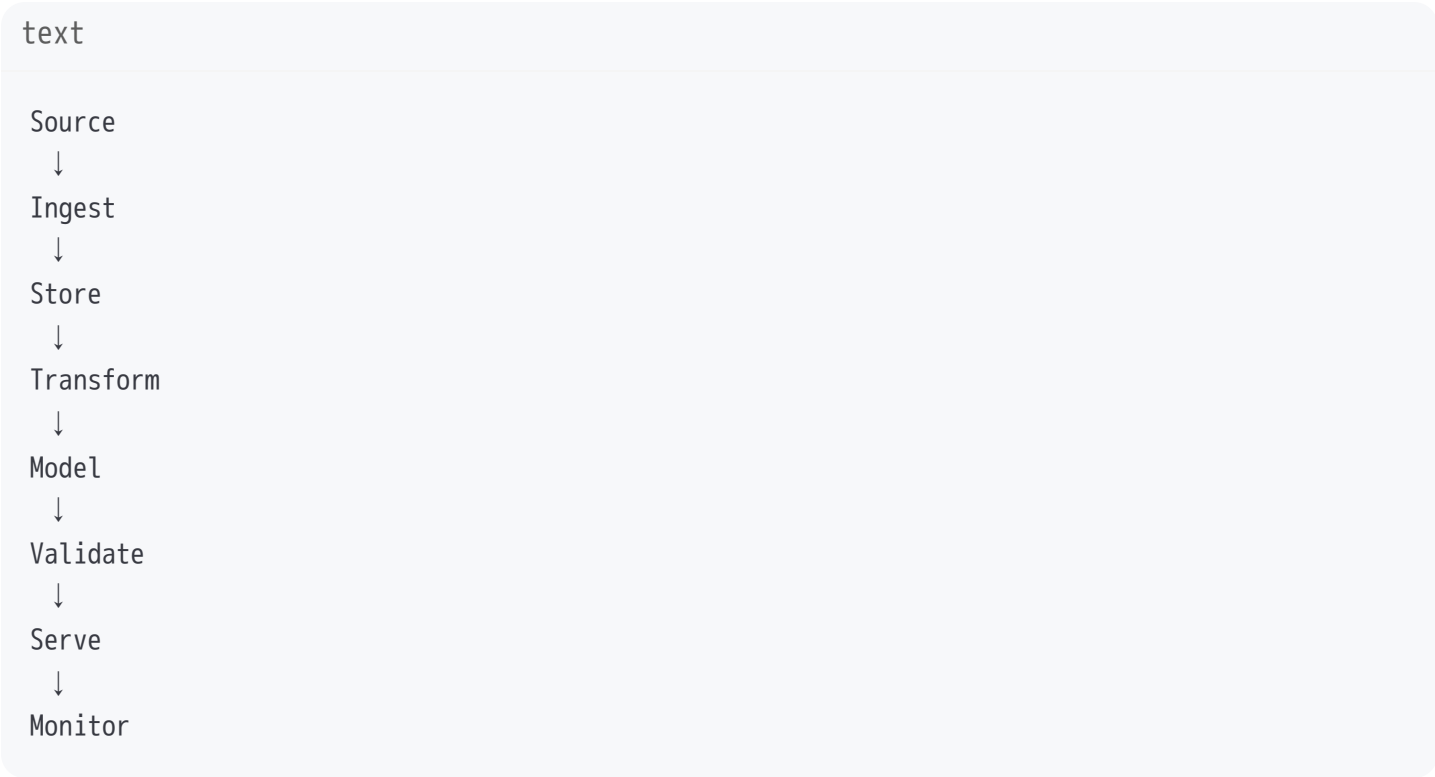


Then show the supporting concepts around it:





And finally:



That becomes your **complete Data Engineer mental model**.

3. The connections between every video

This is the part you were asking for.

Video	Learn	Why next topic makes sense
1	Data journey	Need to know where data lives
2	DB/Lake/Warehouse	Need to understand workloads
3	OLTP vs OLAP	Explains operational vs analytical systems
4	Data Warehouse	Need warehouse design
5	Data Modeling	Need table design
6	Fact vs Dimension	Core warehouse tables

Video	Learn	Why next topic makes sense
7	Star Schema	Connect facts + dimensions
8	Snowflake	Alternative modeling approach
9	SCD	Dimensions change
10	SCD 1 vs 2	Implement dimension history
11	ETL vs ELT	Need to load warehouse
12	Batch vs Streaming	Different ingestion patterns
13	Ingestion	How data enters platform
14	ADF	Azure implementation
15	ADF components	Build reusable pipelines
16	Full vs Incremental	Avoid reprocessing
17	Watermark	Detect changes
18	CDC	Better change detection
19	Metadata	Scale ingestion
20	Lakehouse	Modern storage architecture
21	Medallion	Organize lakehouse
22	Spark	Process big data
23	Partitions	Understand Spark execution
24	Shuffle	Understand performance
25	Joins/Broadcast	Optimize Spark
26	Delta Lake	Reliable lakehouse tables
27	Data Quality	Ensure correctness
28	Security/Governance	Make platform enterprise-ready
29	Monitoring	Operate production pipelines
30	End-to-End	Connect everything

This is much stronger than a list like:

ADF → Spark → SQL → Kafka → Snowflake → Power BI → Databricks...

because that jumps between layers without teaching **why the next concept exists**.

4. And every individual Reel should follow this pattern

For your actual scripts, I would now use this exact structure:

1. HOOK

0–3 sec

Create a real engineering problem.

“Why are you processing 100 million rows when only 10,000 changed?”

2. PROBLEM

3–8 sec

Show what goes wrong.

text

```
100M rows
↓
Every day
↓
Slow + expensive
```

3. CONCEPT

8–18 sec

Introduce the solution.

“This is why Data Engineers use incremental loading.”

4. VISUAL EXPLANATION

18–30 sec

Don't show paragraphs.

Show:

text

```
Last Run
10:00 AM
↓
updated_at > 10:00
↓
```

5. REAL ENGINEERING USE

30–38 sec

“ADF can use this pattern to move only newly inserted or updated records.”

6. DECISION RULE

38–43 sec

“Small changes? Incremental load.
Everything needs rebuilding? Full load.”

7. CONNECTION TO NEXT VIDEO

Don't necessarily say this aloud, but design the next Reel around the unanswered question:

“But how does the pipeline know what changed?”

Next Reel → Watermark.

This is the approach I recommend for your entire series

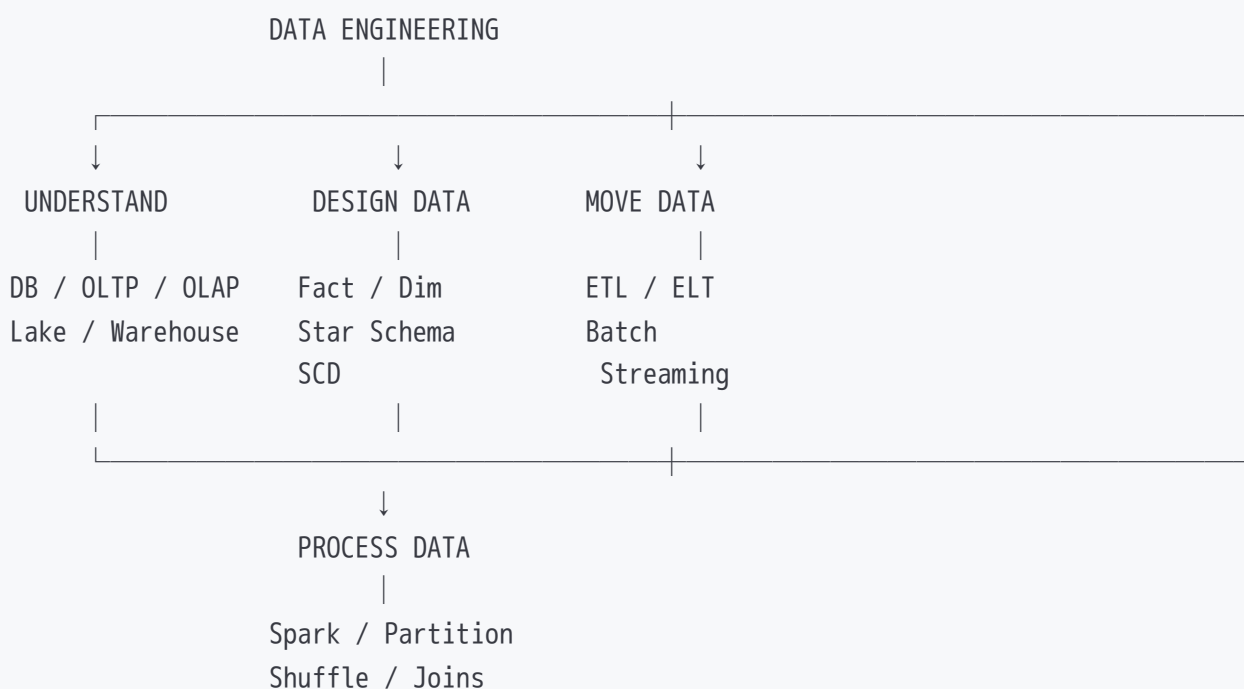
Not:

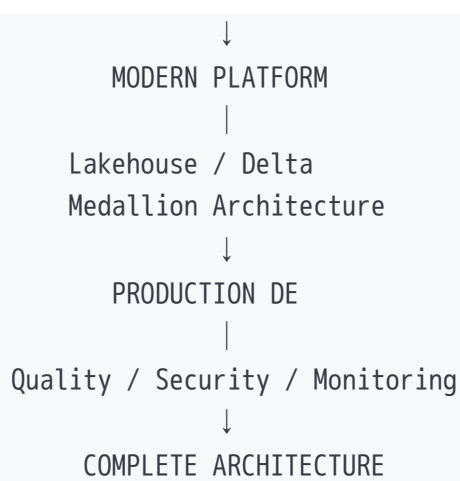
30 concepts = 30 definitions

But:

30 concepts = one Data Engineering story.

text



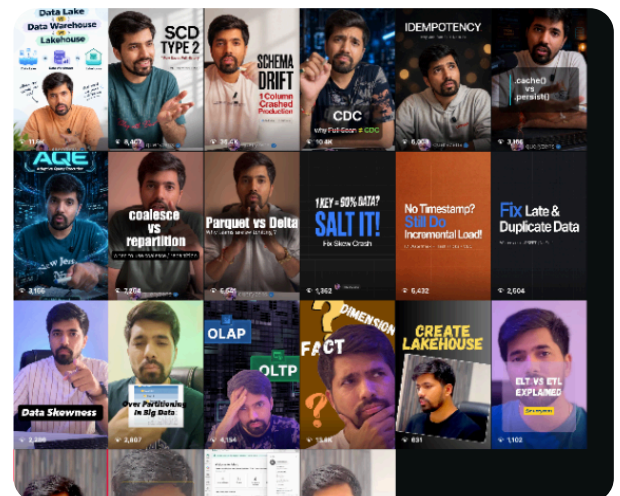


This is the structure I would use going forward. It gives your audience a reason to watch the next video because every concept solves a problem introduced by the previous one.

One limitation: I can inspect the uploaded video's visuals and timing here, but I don't have a speech-transcription engine available in this runtime, so I won't pretend I have reproduced the reference creator's **word-for-word audio**. The pattern above is based on the actual visible sequence and content; for your own videos, we can make the scripts from this structure rather than falsely claiming an exact audio transcript.

2026-10-02 12:35:00

You 



understand images and heading of topics .THinks like adata engineer content create and connectiong concept cover from proper mannerr . for example data lake ,lakhouse,data warehouse all are releted so need to cover at once .

 ChatGPT gpt-5-6

Exactly. **Now I understand what you mean.** The screenshot changes the requirement significantly.

You are not asking me to make a generic “30 DE topics” list. You want me to study the **type of topics this creator chooses**, identify which concepts are connected, and then create a **proper Data Engineering content map where related concepts are taught together**.

For example, you correctly pointed out:

Data Lake vs Data Warehouse vs Lakehouse

These should **not** be three separate random reels because the viewer needs to understand their relationship and differences together.

Likewise:

CDC → Incremental Load → Watermark → Idempotency → Late/Duplicate Data

are strongly connected and should form a content cluster.

And:

Partitioning → Data Skew → Salt → Repartition → Coalesce → Broadcast/Join

are another connected Spark-performance cluster.

That is the structure we should use.

1. What I see in this creator's content

From the screenshot, the topics are not beginner-only definitions.

They are mostly “**engineering decision**” topics.

Examples:

Architecture / storage

- Data Lake vs Data Warehouse vs Lakehouse
- Create Lakehouse

Data modeling

- Fact vs Dimension
- SCD Type 2
- Schema Drift

Data movement

- CDC
- Incremental Load

- Late & Duplicate Data
- Idempotency

Spark performance

- AQE
- Coalesce vs Repartition
- Data Skew
- Salting
- Over Partitioning
- Parquet vs Delta

Query / application performance

- Cache vs Persist

Fundamentals

- OLTP vs OLAP
- ETL vs ELT

So the creator is mixing:

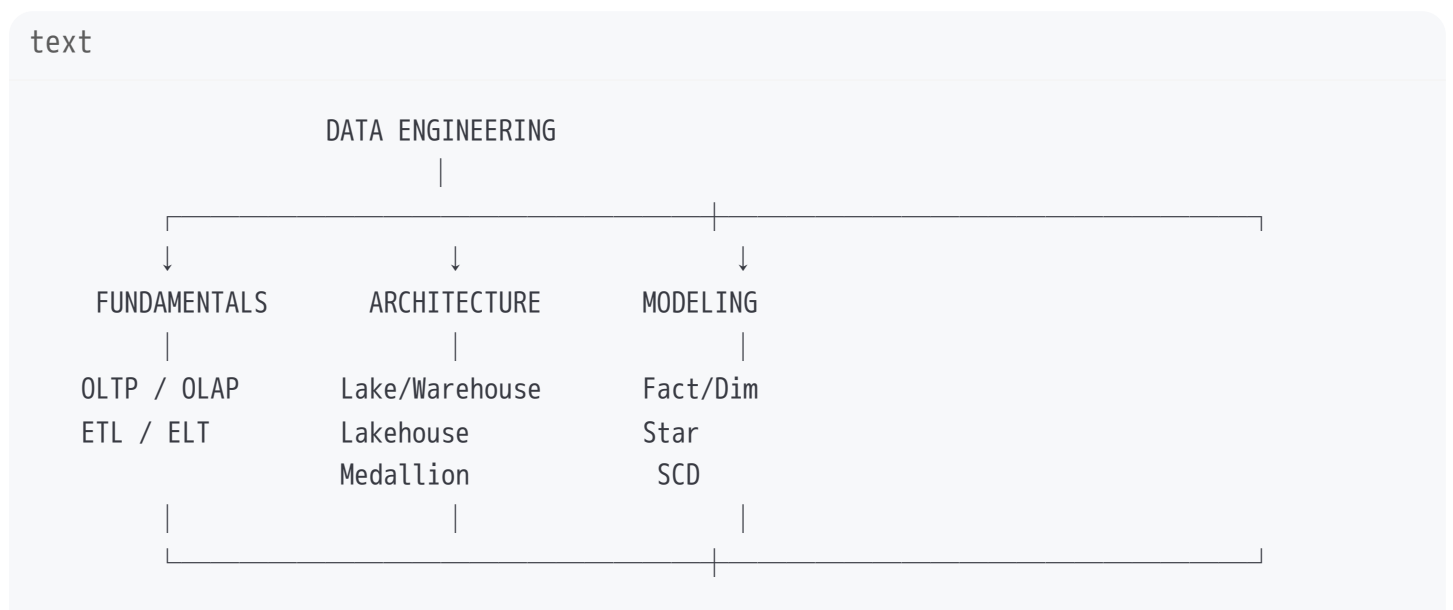
Fundamentals + architecture + real production problems + optimization.

That is exactly what your series should do.

2. The biggest change: don't make 30 isolated topics

Instead, create **30 videos across connected concept clusters**.

Think of the series like this:





Now **each cluster has an internal story.**

3. The 30 topics I would actually use

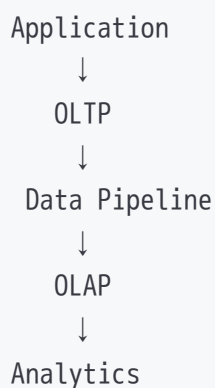
CLUSTER 1 – DATA FUNDAMENTALS

01. OLTP vs OLAP

Start here because this establishes:

Operational system vs analytical system

text



This gives context for almost everything later.

02. ETL vs ELT

Once they understand OLAP:

“How does data actually get from the operational world into the analytical world?”

text

ETL

Source → Transform → Warehouse

ELT

Source → Warehouse → Transform

03. Batch vs Streaming

Now introduce different data movement patterns.

text

Batch

Data → Wait → Process

Streaming

Event → Process → Result

This creates the foundation for ingestion.

CLUSTER 2 — STORAGE ARCHITECTURE

These **three absolutely belong together**, as you pointed out.

04. Data Lake vs Data Warehouse vs Lakehouse

One complete video.

Not:

✗ Data Lake video

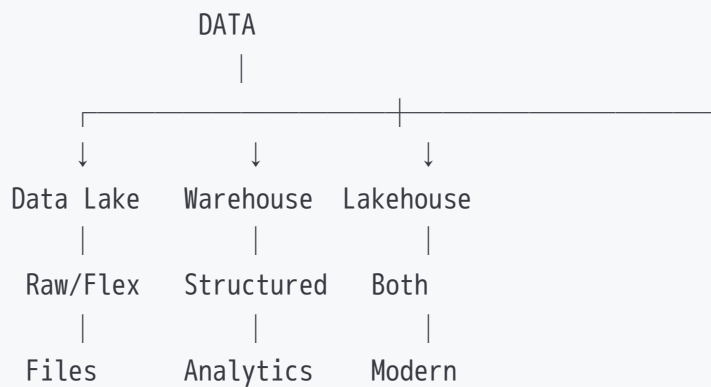
✗ Warehouse video

✗ Lakehouse video

Instead:

ONE comparison:

text



Then answer:

“Which one should I use and why?”

This is exactly the type of content your reference creator makes.

05. Medallion Architecture

Once the viewer understands Lakehouse:

text



Now Bronze/Silver/Gold has context.

06. Parquet vs Delta Lake

Now ask:

“If the Lakehouse stores files, why do modern platforms use Delta?”

Compare:

text

Parquet
→ File format

Delta
→ Table/storage layer
→ ACID
→ MERGE
→ UPDATE
→ DELETE
→ Time Travel

This connects directly to later CDC and incremental loading.

CLUSTER 3 – DATA MODELING

07. Fact Table vs Dimension Table

Your reference topic.

Teach:

text

FACT
What happened?

DIMENSION
Who / What / Where / When?

Then practical decision rule:

text

SUM / AVG
→ FACT

FILTER / GROUP BY
→ DIMENSION

08. Star Schema vs Snowflake Schema

Now the viewer already knows Fact + Dimension.

So:

text

STAR



versus:

text

SNOWFLAKE



09. SCD Type 1 vs Type 2

Now the viewer asks:

“What happens when dimension data changes?”

Then:

text

Type 1
Old → overwrite

Type 2
Old → preserve
New → create version

10. SCD Type 2 in Real Pipeline

Don't stop at definition.

Show:

text

```
Source
  ↓
Detect Change
  ↓
Expire Old Record
  ↓
Insert New Version
  ↓
Delta MERGE
```

This connects:

SCD → CDC → Delta

which is much more useful.

CLUSTER 4 – INCREMENTAL DATA ENGINEERING

This is one of the strongest clusters from the screenshot.

11. Full Load vs Incremental Load

Start:

“Why process 100 million rows when only 10,000 changed?”

text

```
FULL
100M → Process
```

```
INCREMENTAL
10K → Process
```

12. Watermark-Based Incremental Load

Then answer:

“How do I know which rows are new?”

text

```
Last Watermark
  ↓
updated_at > watermark
  ↓
```

13. CDC — Change Data Capture

Now ask:

“What if updated_at doesn't reliably tell me everything?”

Then:

text

```
INSERT
UPDATE
DELETE
  ↓
  CDC
  ↓
Target
```

Compare:

Watermark vs CDC

This should be one video.

14. Incremental Load Without Timestamp

This is directly inspired by the screenshot:

“No Timestamp? Still Do Incremental Load!”

Teach alternatives:

text

```
ID Watermark
Hash
CDC
Modified sequence
Change tracking
```

This is a very good practical DE topic.

15. Idempotency

Now the natural problem:

“What happens if your pipeline runs twice?”

text

Run 1
100 records

Run 2
100 records again

✗ Duplicate

Idempotent design:

text

Run 1 → 100
Run 2 → Still 100

Core rule:

Running the same pipeline twice should not corrupt the result.

16. Late Arriving & Duplicate Data

Now connect:

Incremental loading + real-world data problems

Examples:

text

Late Event
10:05 event arrives at 10:30

and:

text

Same Event
A
A

Teach:

- deduplication
- event time

- ingestion time
- watermark
- upsert

This cluster now becomes:

text

```
Incremental
  ↓
Watermark
  ↓
CDC
  ↓
Idempotency
  ↓
Late/Duplicate Data
```

That's a **real engineering story**.

CLUSTER 5 – SCHEMA & DATA RELIABILITY

17. Schema Evolution vs Schema Drift

The screenshot specifically has:

SCHEMA DRIFT – 1 Column Crashed Production

Teach the distinction:

text

Schema Evolution
Expected change

Schema Drift
Unexpected change

Example:

text

Before:
customer_id
name
age

After:

```
customer_id
name
age
email
```

Then explain how pipelines react.

18. Data Quality Checks

Now:

“Even if the schema is correct, can the data still be wrong?”

Check:

text

```
NULL
DUPLICATE
WRONG TYPE
INVALID FK
ROW COUNT
FRESHNESS
```

Important concept:

Pipeline success $\neq$ data quality success.

CLUSTER 6 – SPARK FUNDAMENTALS

Now we enter the screenshot's heavy Spark section.

19. Why Spark?

Start:

“What happens when your data becomes too large for one machine?”

text

```
1 Machine
  ↓
Too much data

Spark
  ↓
Cluster
  ↓
```

20. Spark Partitioning

Then explain:

text

Dataset



Partition 1

Partition 2

Partition 3

Partition 4



Parallel Tasks

This is the foundation for every next Spark-performance topic.

21. Over-Partitioning vs Under-Partitioning

This directly matches the screenshot.

Explain:

text

Too Few

→ Large partitions

→ Low parallelism

Too Many

→ Too many tasks

→ Scheduling overhead

Then explain how to think about partition count.

22. Data Skew

Now:

“What if one partition gets 90% of the data?”

text

P1 → 5%

P2 → 5%

P3 → 5%

P4 → 85% ❌

This connects directly to partitioning.

23. Salting

Now solve the skew problem.

```
text
```

```
Skewed Key
```

```
Customer = UNKNOWN
```

Instead:

```
text
```

```
UNKNOWN_1
```

```
UNKNOWN_2
```

```
UNKNOWN_3
```

```
UNKNOWN_4
```

Then distribute the workload.

This is exactly the kind of **problem** → **solution** content shown in the screenshot.

24. Repartition vs Coalesce

Now that partitioning is understood:

```
text
```

```
REPARTITION
```

```
→ Increase/decrease partitions
```

```
→ Shuffle
```

```
COALESCE
```

```
→ Usually decrease partitions
```

```
→ Less/no full shuffle
```

Then decision rule:

Need redistribution? → Repartition

Just reducing partitions? → Coalesce

CLUSTER 7 – SPARK PERFORMANCE

25. Shuffle

This should come before advanced optimization.

Teach why:

text

JOIN
GROUP BY
ORDER BY
DISTINCT

can cause:

text

Partition
↓
SHUFFLE
↓
Network transfer
↓
New partition

And why shuffle is expensive.

26. Broadcast Join

Now solve one common join problem:

text

Huge Fact
+
Small Dimension

Instead of moving the huge table:

text

Broadcast small table
↓
Workers

This connects directly to your **Fact vs Dimension** video from much earlier.

That's exactly the kind of cross-topic connection you want.

27. AQE — Adaptive Query Execution

Now introduce:

“What if Spark could adjust the execution plan based on what actually happens during the job?”

Teach high level:

text

```
graph TD
    A[Original Plan] --> B[Runtime Statistics]
    B --> C[Adaptive Plan]
```

Cover:

- coalescing shuffle partitions
- skew handling
- join strategy changes

CLUSTER 8 — PRODUCTION ENGINEERING

28. Cache vs Persist

Now teach when repeatedly using the same DataFrame:

text

```
graph TD
    subgraph Without_cache [Without cache]
        direction TB
        A[Read] --> B[Compute]
        B --> C[Read]
        C --> D[Compute]
        D --> E[Read]
        E --> F[Compute]
    end
```

versus:

text

Compute



Cache/Persist



Reuse

And importantly:

Cache is not automatically faster.

This keeps it engineering-focused.

29. Production Pipeline Reliability

Combine the operational concepts:

text

Retry

Idempotency

Checkpoint

Logging

Monitoring

Alerting

Failure handling

Show:

text

Pipeline



Failure



Retry



Same input



No duplicates



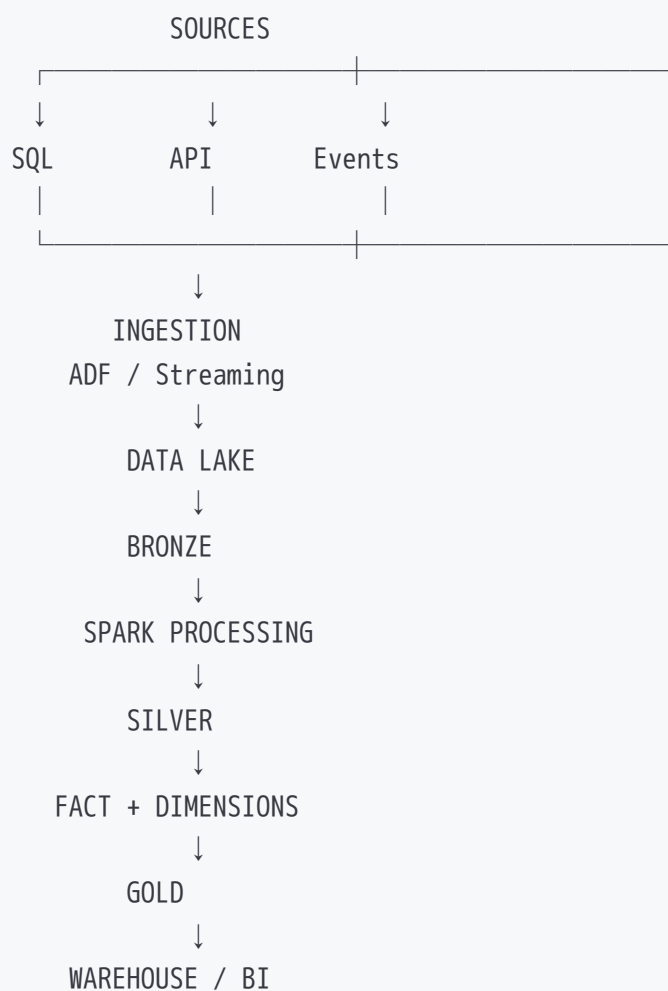
Continue safely

This connects back to **Idempotency**.

30. COMPLETE DATA ENGINEERING SYSTEM

Final video should connect everything:

text



Supporting engineering:

text

Incremental Load

+

CDC

+

Watermark

+

Idempotency

+

Schema Evolution

+

Data Quality

+

Partitioning

+

Skew Handling

+

AQE

Now the audience can see **where every previous video fits**.

4. The most important connections

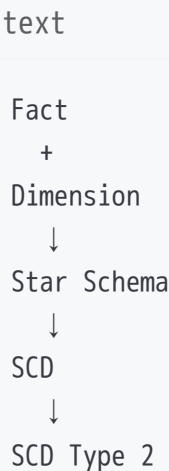
This is how I would deliberately connect your Reels.

Storage connection



Videos 4–6

Modeling connection



Videos 7–10

Incremental connection

text

Full Load
↓
Incremental
↓
Watermark
↓
CDC
↓
No Timestamp
↓
Idempotency
↓
Late/Duplicate Data

Videos 11–16

This is one of the strongest sequences.

Spark performance connection

text

Spark
↓
Partition
↓
Over Partition
↓
Skew
↓
Salt
↓
Repartition / Coalesce
↓
Shuffle
↓
Broadcast
↓
AQE

Videos 19–27

This is another very strong sequence.

5. And this is how your content should feel

Instead of:

✗ Beginner-style

"Today we are going to learn what CDC is. CDC stands for Change Data Capture. It is a technique..."

Your style should be:

✓ Reference-style

"Why scan the entire table when you only need the rows that changed?"

"Imagine your customer table has 500 million records. Yesterday, only 20,000 records were updated."

"A full scan means you're reading all 500 million again."

"CDC changes the approach."

"Instead of asking the database, *'Which rows look different?'*, CDC captures the actual INSERT, UPDATE and DELETE operations."

"Full scan finds changes. CDC captures changes."

That is the content style visible in the screenshot:

Problem → curiosity → technical explanation → visual → practical decision → memorable rule.

6. One more important thing: topic selection

I would **not** force every traditional DE topic into these 30 videos.

For example, topics like:

- Python basics
- SQL SELECT
- JOIN basics
- GROUP BY
- Azure basics
- What is ADF
- What is Databricks

can be useful for a **beginner course**, but they don't necessarily fit this creator's content style.

The screenshot's content is more like:

“Here is a real problem Data Engineers face. Here's the concept that solves it. Here's when to use it.”

So your series should target:

DE concepts + engineering decisions + production problems + optimization.

That's much closer to the content you're trying to create.

Final structure

text

```
01-03 Fundamentals
04-06 Storage Architecture
07-10 Data Modeling
11-16 Incremental / CDC / Reliability
17-18 Schema & Data Quality
19-24 Spark Fundamentals + Partitioning
25-27 Spark Optimization
28-29 Production Engineering
30 Complete DE Architecture
```

And yes — **Data Lake vs Data Warehouse vs Lakehouse should be ONE connected video**, not three separate videos. Similarly, **CDC, watermarking, incremental loading, idempotency, and late/duplicate data should be treated as a connected storyline**, and the Spark topics should build from **partitioning → skew → salting → repartition/coalesce → shuffle → broadcast → AQE**. That is the kind of connection-based curriculum you were asking for.

2026-10-02 12:36:40 You 

cover it all basic DE concpets in 20 ,15 video

 ChatGPT gpt-5-6

Yes. If your goal is “**cover all the important basic Data Engineering concepts**” while keeping the **same problem → explanation → comparison → decision-rule style** as the reference creator, then I would reduce it to **20 videos**.

And if you want an even tighter series, I'll give you the **15-video version** below.

The key is: **related concepts stay together** instead of splitting them into unnecessary separate videos.

OPTION A – 20 VIDEO DATA ENGINEERING SERIES

This is the version I recommend because it gives enough space to cover the fundamentals properly.

PHASE 1 – DATA ENGINEERING FOUNDATION

VIDEO 01 – What Does a Data Engineer Actually Do?

Cover:

- What is data engineering?
- Data sources
- Data ingestion
- Storage
- Transformation
- Data modeling
- Serving/analytics
- Monitoring

Complete flow:

text

```
graph TD
    Applications --> Databases[\"Databases / APIs / Files\"]
    Databases --> Ingestion
    Ingestion --> Storage
    Storage --> Transformation
    Transformation --> DataWarehouse[\"Data Warehouse / Lakehouse\"]
    DataWarehouse --> BI[\"BI / Analytics / ML\"]
```

Memory rule:

Data Engineers build the systems that make data usable.

VIDEO 02 – Database vs Data Warehouse vs Data Lake vs Lakehouse

Put all related storage concepts **in one video**.

Cover:

Database

Operational applications

Data Warehouse

Structured analytical data

Data Lake

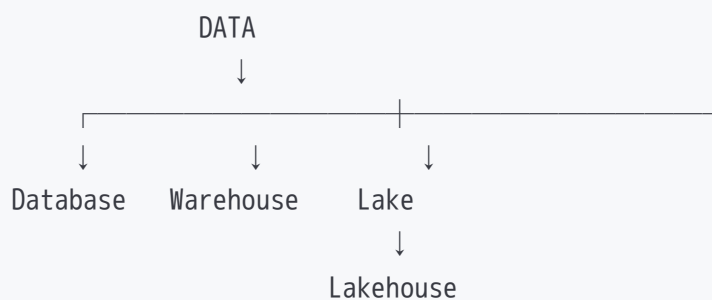
Raw + large + varied data

Lakehouse

Lake flexibility + warehouse capabilities

Visual:

text



Then answer:

When would I use each one?

This is much better than making four separate videos.

VIDEO 03 – OLTP vs OLAP + ETL vs ELT

These concepts are strongly connected.

OLTP

text

Application
Transactions
INSERT
UPDATE
DELETE

OLAP

text

Analytics
Aggregation
Historical analysis
Large queries

Then:

ETL

text

Extract
↓
Transform
↓
Load

ELT

text

Extract
↓
Load
↓
Transform

End with:

OLTP runs the business. OLAP analyzes the business. ETL/ELT moves data between systems.

PHASE 2 — DATA MODELING

VIDEO 04 — Data Modeling + Normalization vs Denormalization

Cover:

- What is data modeling?
- Entities
- Relationships
- Primary key
- Foreign key

- Normalization
- Denormalization

Example:

text

```
Customer
|
Order
|
Product
```

Then explain why analytical systems often use more denormalized designs.

This prepares the viewer for Fact/Dimension.

VIDEO 05 — Fact Table vs Dimension Table + Star Schema

One connected video.

Don't separate these.

text

```
      DIM CUSTOMER
      |
DIM PRODUCT - FACT SALES - DIM DATE
      |
      DIM STORE
```

Explain:

Fact

What happened?

- sales
- quantity
- revenue
- transactions

Dimension

Who / What / Where / When?

- customer

- product
- date
- location

Then:

Star Schema = Fact in the center + Dimensions around it.

Decision rule:

Facts measure events. Dimensions describe events.

VIDEO 06 – Star Schema vs Snowflake + SCD Type 1 & 2

Now continue naturally.

Star

text

Dimension



Fact



Dimension

Snowflake

text

Fact



Product



Category



Department

Then introduce the next problem:

“But what happens when a dimension changes?”

SCD Type 1

text

Pune → Mumbai

Overwrite.

SCD Type 2

text
Pune 2024 → 2025
Mumbai 2025 → Current

Preserve history.

PHASE 3 – DATA INGESTION

VIDEO 07 – Data Ingestion: Batch vs Streaming

Cover:

Batch

text
Collect ↓ Process ↓ Load

Example:

Daily sales.

Streaming

text
Event ↓ Process ↓ Result

Example:

Fraud detection.

Also explain common sources:

- SQL
- APIs
- CSV
- JSON
- SaaS
- events

VIDEO 08 – Full Load vs Incremental Load

Start with:

“Why process 100 million rows when only 10,000 changed?”

Full Load

text

100M → Process

Incremental

text

10K → Process

Cover:

- first load
- subsequent loads
- efficiency
- cost
- performance

Then naturally introduce:

“But how do we know which 10,000 records changed?”

VIDEO 09 – Watermark vs CDC

These absolutely belong together.

Watermark

```
text

Last processed:
10:00 AM

WHERE updated_at > 10:00
```

CDC

```
text

INSERT
UPDATE
DELETE
↓
CDC
```

Comparison:

Watermark	CDC
Uses a column/value	Captures actual changes
Timestamp/ID common	Insert/update/delete
Simple	More comprehensive
Can miss certain changes	Designed for change capture

End:

Watermark tracks a position. CDC tracks changes.

VIDEO 10 – Idempotency + Duplicate + Late Data

This is a very important production concept.

Problem:

```
text

Pipeline runs twice

100 records
```

+
100 records

= 200 ❌

Idempotent:

text

Run 1 → 100
Run 2 → still 100

Then connect:

- duplicate records
- retries
- late-arriving data
- upsert
- deduplication

Memory rule:

A retry should not create a different final result.

PHASE 4 — AZURE / MODERN DATA PLATFORM

VIDEO 11 — ADF Fundamentals

Cover the essential ADF components together:

text

ADF
|
├── Pipeline
├── Activity
├── Linked Service
├── Dataset
├── Trigger
└── Integration Runtime

Explain each in one practical example.

Example:

text

```
SQL Server
↓
Linked Service
↓
Dataset
↓
Copy Activity
↓
ADLS
```

VIDEO 12 — Parameters vs Variables + Dynamic Content + Metadata-Driven Pipeline

These belong together because they solve the same problem:

How do I avoid hardcoding pipelines?

Parameter

Input to pipeline.

Variable

Runtime value.

Dynamic content

Build values dynamically.

Metadata-driven

One pipeline → many tables.

```
text

Metadata
↓
Generic Pipeline
↓
ForEach
↓
100 Tables
```

This is much more valuable than making four beginner videos.

VIDEO 13 — Data Lake / Lakehouse Architecture + Medallion

Now connect the storage concepts to an actual implementation.

text

SOURCE



BRONZE

Raw



SILVER

Clean



GOLD

Business Ready

Explain:

- raw layer
- cleaned layer
- business layer
- why separation matters

Then connect:

text

Lakehouse



Bronze / Silver / Gold

VIDEO 14 – Parquet vs Delta Lake

Cover:

Parquet

File format.

Delta

Table/storage layer built around files with additional capabilities.

Explain:

- ACID transactions
- schema enforcement
- schema evolution

- UPDATE
- DELETE
- MERGE
- time travel

Then connect Delta to:

Incremental Load + SCD + CDC

PHASE 5 — BIG DATA / SPARK

VIDEO 15 — Apache Spark: Architecture + DataFrame + Lazy Evaluation

Cover the basic Spark concepts together.

text

```
graph TD;
  Driver --> Cluster;
  Cluster --> Executors;
  Executors --> Tasks;
```

Then:

DataFrame

and:

Lazy evaluation

Example:

text

```
graph TD;
  Read --> Filter;
  Filter --> Select;
  Select --> Group;
  Group --> Action;
```

Explain:

VIDEO 16 – Spark Partitioning + Shuffle

These must be taught together.

Partition

text

Data



P1

P2

P3

P4

Parallel processing.

Then:

Shuffle

text

P1 —

P2 —+→ SHUFFLE → New partitions

P3 —

P4 —

Explain why shuffle is expensive.

Cover operations such as:

- join
- groupBy
- orderBy
- distinct

VIDEO 17 – Repartition vs Coalesce + Data Skew + Salting

This is your Spark optimization cluster.

Repartition

Redistributes data.

Coalesce

Usually reduces partitions with less movement.

Then:

Data Skew

text

P1 → 5%
P2 → 5%
P3 → 5%
P4 → 85% ❌

Then:

Salting

Split the hot key:

text

UNKNOWN
↓
UNKNOWN_1
UNKNOWN_2
UNKNOWN_3
UNKNOWN_4

This gives a complete story:

Partition → imbalance → skew → solution.

VIDEO 18 — Spark Joins + Broadcast Join + AQE

Cover:

Normal large-large join

Potentially expensive shuffle.

Broadcast

text

Huge Fact
+
Small Dimension



Broadcast Dimension

Then:

AQE

Spark can adjust execution using runtime information.

Cover high-level:

- shuffle partition coalescing
- skew handling
- join strategy adaptation

PHASE 6 — PRODUCTION DATA ENGINEERING

VIDEO 19 — Schema Evolution + Data Quality + Data Validation

First:

Schema Evolution

Expected change.

Schema Drift

Unexpected change.

Example:

text

Before

id | name | age

After

id | name | age | email

Then:

Data Quality

text

NULL?

DUPLICATE?

WRONG TYPE?

MISSING RECORD?



WRONG COUNT?
FRESH DATA?

Important line:

Pipeline SUCCESS does not always mean Data SUCCESS.

VIDEO 20 – Production Data Pipeline: Monitoring + Security + Reliability + Complete Architecture

Final video combines everything.

Reliability

- retry
- failure handling
- idempotency
- checkpointing

Monitoring

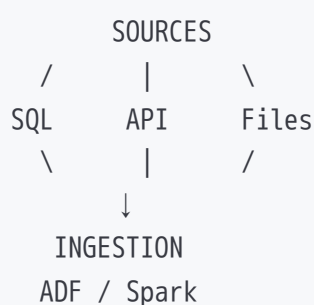
- logs
- metrics
- alerts
- pipeline status

Security

- RBAC
- secrets
- access control
- sensitive data

Then complete architecture:

text





Supporting everything:

text

Incremental Load
 Watermark
 CDC
 Idempotency
 Delta
 Data Quality
 Monitoring
 Security

Final line:

“A Data Engineer doesn't just move data. We build a reliable system that makes data trustworthy and usable.”

OPTION B – COMPRESSED 15-VIDEO VERSION

If you want only **15 Reels**, I would combine them like this.

#	Video	Main concepts
0 1	What Does a Data Engineer Do?	DE role, data lifecycle, pipeline
0 2	OLTP vs OLAP + ETL vs ELT	Operational vs analytical, ETL/ELT
0 3	Data Lake vs Warehouse vs Lakehouse	All 3 + when to use
0 4	Data Modeling: Fact, Dimension & Star Schema	Modeling + fact/dim + star

#	Video	Main concepts
05	Snowflake Schema + SCD Type 1 & 2	Snowflake + history
06	Batch vs Streaming + Data Ingestion	Sources + ingestion patterns
07	Full vs Incremental + Watermark	Incremental fundamentals
08	CDC + No Timestamp + Idempotency	Change detection + reliability
09	ADF: Pipeline + Linked Service + Dataset + IR	Core ADF
10	Parameters + Dynamic Content + Metadata Pipelines	Reusable ADF
11	Medallion + Parquet vs Delta	Lakehouse implementation
12	Spark Fundamentals + DataFrames + Lazy Evaluation	Spark foundation
13	Partition + Shuffle + Repartition + Coalesce	Spark execution
14	Data Skew + Salting + Broadcast + AQE	Spark optimization
15	Data Quality + Monitoring + Security + End-to-End Architecture	Production DE

Which one fits your goal better?

15 videos

Good if your goal is:

“Give people a complete DE roadmap quickly.”

text

15 videos

↓

High information density

↓
Faster series

But some videos become heavier.

20 videos

Better if your goal is:

“Build a serious DE educational content series.”

text

20 videos
↓
Better explanation
↓
Better examples
↓
Better visual storytelling
↓
Less concept overload

For your **reference-style content**, I would use the **20-video version**.

Most important: the sequence is now connected

Your series would feel like one story:

text

01 DE
↓
02 OLTP / OLAP
↓
03 Lake / Warehouse / Lakehouse
↓
04 Modeling
↓
05 Fact / Dimension / Star
↓
06 SCD
↓
07 Ingestion
↓
08 Incremental
↓
09 Watermark / CDC
↓

```
10 Idempotency
  ↓
11 ADF
  ↓
12 Dynamic / Metadata
  ↓
13 Medallion
  ↓
14 Delta
  ↓
15 Spark
  ↓
16 Partition / Shuffle
  ↓
17 Skew / Salt / Repartition
  ↓
18 Join / Broadcast / AQE
  ↓
19 Schema / Quality
  ↓
20 Production Architecture
```

This is the key difference from the earlier list: **every topic has a reason to appear after the previous topic.**

And for each video, we'll use your reference creator's content formula:

Strong problem/hook → why it matters → simple visual → concept → real DE example → compare alternatives → “when should I use this?” → memorable one-line rule.

That gives you **basic DE coverage without turning the series into 20 textbook-definition videos.**

2026-10-02 12:40:33 You 

So as per i shared videos you know the script how to explain concept so as it at once script for 1 topic give me .proper think how to prep to explain this concpet with good example and all .then give me entire script only no need extra information . make sure as it is that videos teaching technique like generate script and make sure more human writetern by own script .not looinng like a ai generate scrip and onyl human read that

DATA LAKE vs DATA WAREHOUSE vs LAKEHOUSE – SCRIPT

“Data Lake, Data Warehouse aur Lakehouse...
teeno data store karte hain.

Toh phir actual difference kya hai?

Chalo ek simple real-world example se samajhte hain.

Suppose ek e-commerce company hai.

Uske paas customer data, orders, product data, website logs, CSV files, APIs... basically har type ka data aa raha hai.

Ab ye raw data exactly as it is store karna hai?

Data Lake.

Yahan aap structured, semi-structured, even unstructured data bhi store kar sakte ho.

Lekin problem ye hai —
analytics ke liye raw data directly use karna messy ho sakta hai.

So business reporting ke liye hum use karte hain...

Data Warehouse.

Warehouse mein data properly cleaned, structured aur business queries ke liye optimized hota hai.

Ab problem ye aayi...

Lake flexible hai, Warehouse structured hai.

Toh agar mujhe dono capabilities ek hi platform mein chahiye?

That's where Lakehouse comes in.

Lakehouse gives you the flexibility of a Data Lake,
but adds warehouse-like capabilities such as reliable tables, transactions and analytics.

So simple yaad rakho—

Raw aur flexible data? → Data Lake.

Structured business analytics? → Data Warehouse.

Lake ki flexibility + Warehouse capabilities? → Lakehouse.

Aur isi liye modern Data Engineering platforms mein Lakehouse architecture itna common ho gaya hai.”

i mean check once videos i shared audion and understand their technique technique . give first concept with proper only script at once .

 ChatGPT gpt-5-6-t-mini

“Data Lake, Data Warehouse, aur Lakehouse — naam teen hain, but actually confusion ek hi hai...

Data ko rakhna kahan hai?

Suppose ek e-commerce company hai.

Har din data aa raha hai —
orders database se,
customer data CRM se,
website logs JSON mein,
aur kuch CSV files bhi.

Ab ye sab data hum ek jagah rakhna chahte hain.

Yahin pe Data Lake ka use hota hai.

Jo bhi data hai, raw form mein rakh do.
Structured ho, JSON ho, logs ho — flexibility bahut hai.

But now business bolta hai —
“Bhai, mujhe last 2 years ka sales report chahiye.”

Raw files ke upar directly analytics karna messy ho sakta hai.

So we use a **Data Warehouse**.

Yahan data clean, structured aur analytics ke liye ready hota hai.

Ab interesting part...

Lake flexible hai.
Warehouse analytics ke liye structured hai.

But what if I want both?

Lake ki storage aur flexibility bhi chahiye,
aur Warehouse jaise table management, transactions aur analytics capabilities bhi?

That's where Lakehouse comes in.

So ek simple rule yaad rakho—

Raw, flexible, any type of data → Data Lake.

Clean, structured, business analytics → Data Warehouse.

Lake ki flexibility + Warehouse capabilities → Lakehouse.

Bas isi difference ko samajh lo,
modern data architecture ka bahut saara confusion automatically clear ho jayega.”

2026-10-02 13:16:30

You



give next



ChatGPT gpt-5-6-t-mini

“Data warehouse bana liya...
but ek important question hai—

warehouse ke andar tables kaise design karoge?

Because agar data properly model nahi kiya,
toh kal ko ek simple business query bhi complicated ho jayegi.

Ek simple example lete hain.

Maan lo hamare paas ek e-commerce company hai.

Customer hai,
Product hai,
aur Orders hain.

Customer ka ek order ho sakta hai,
aur ek customer ke multiple orders bhi ho sakte hain.

So hum in sabko bas ek giant table mein daal dein?

Technically kar sakte ho...

but jaise-jaise data grow karega,
same customer aur product information baar-baar repeat hogi.

Yahin pe aata hai **Data Modeling**.

Data modeling basically decide karta hai—

kaunsa data kis table mein jayega,
tables ek-doesre se kaise related hongy,

aur analytics ke liye data ko kaise organize karna hai.

Aur yahan pe do important approaches aati hain.

Normalization — data ko smaller related tables mein break karo, taaki duplication kam ho.

Denormalization — related data ko combine karo, taaki analytical queries simpler aur faster ho sakein.

So simple way mein yaad rakho—

Normalization → less redundancy.

Denormalization → easier analytics.

Aur isi modeling ke basis par hum next decide karte hain—

Fact table kya hogi, Dimension table kya hogi, aur dono connect kaise hongii.”

2026-10-02 13:17:15 You 

what this no sense give proper topic name as per 20 days plan and clear give script only

 ChatGPT gpt-5-6-t-mini

Day 3 — OLTP vs OLAP + ETL vs ELT

“OLTP aur OLAP...

dono database se related hain,
but dono ka kaam bilkul same nahi hai.

Ek simple example lete hain.

Suppose aap Amazon pe ek order place karte ho.

Order place hua, payment hui, stock update hua...

ye saari activities **OLTP system** handle karta hai.

Because yahan main focus hai

fast transactions — insert, update, delete...
one operation at a time.

Ab company ko ye analyse karna hai ki

last 12 months mein kaunsa product sabse zyada sell hua,

kis city se maximum orders aaye,
aur revenue kitna generate hua.

Ab hum directly OLTP database pe aisi heavy analytical queries nahi chalana chahenge.

Yahan aata hai **OLAP**.

OLAP ka focus hota hai
large historical data ko analyse karna.

So simple difference—

OLTP → run the business.

OLAP → analyse the business.

But ab ek question—

OLTP mein jo data generate hua,
wo OLAP system tak jayega kaise?

Yahin pe aata hai **ETL aur ELT**.

ETL mein—

Extract → Transform → Load.

Pehle data nikala,
clean aur transform kiya,
phir warehouse mein load kiya.

ELT mein approach thodi different hai—

Extract → Load → Transform.

Pehle data warehouse ya lakehouse mein load karo,
phir wahi powerful platform use karke transform karo.

So yaad rakho—

OLTP tells you where transactions happen.

OLAP tells you where analysis happens.

Aur ETL ya ELT tells you how data moves between them.”

2026-10-02 13:19:12 You 

not that much good and clear good explain and proper . give it
correct human explaining script

Day 3 – OLTP vs OLAP + ETL vs ELT

“Let’s say you’re ordering a phone from an e-commerce app.

The moment you click **Place Order**,

the system has to save your order, update the payment, reduce the stock, and confirm the transaction.

This is **OLTP** – Online Transaction Processing.

OLTP is built for one thing:

handling day-to-day transactions quickly and reliably.

But now the business wants to ask—

‘Last year, which product sold the most?’

‘Which city generated the highest revenue?’

‘What was our monthly sales growth?’

These are analytical questions, and this is where **OLAP** comes in.

OLAP is designed to work with large amounts of historical data and run analytical queries.

So think of it this way—

OLTP runs the business.

OLAP helps you understand the business.

Now one more question—

How does the order data from the OLTP system reach the OLAP system?

That’s where **ETL and ELT** come in.

In **ETL**, you Extract the data, Transform it, and then Load it.

In **ELT**, you Extract the data, Load it into the target platform first, and Transform it there.

So the complete picture is:

OLTP → source system

ETL or ELT → move and prepare the data

OLAP → analyse the data

That’s the basic flow behind a lot of modern data platforms.”



understand this video here explain OLAP and OLTP how this video explain concept and when stop in between sentence proper so as like that tone and script format i want



ChatGPT gpt-5-6-t-mini

Day 3 – OLTP vs OLAP

“OLTP aur OLAP...

dono database se related hain.

But dono ka kaam completely different hai.

Let's say you have an e-commerce application.

A customer places an order...

payment happens...

stock gets updated...

order status changes.

These are day-to-day transactions.

And this is what **OLTP** is designed for.

It handles lots of small operations...

quickly...

and continuously.

Now imagine someone runs a heavy SQL query on this same system.

Millions of rows scan ho rahe hain...

aggregation chal rahi hai...

aur query stuck ho gayi.

Because now you're asking your transaction system to do analytics.

That's where **OLAP** comes in.

OLAP systems are built for analysing large amounts of historical data...

for example, Snowflake or BigQuery.

So the architecture normally looks like this:

Application...

to OLTP database...

then data pipeline...
then OLAP warehouse...
and finally Power BI.

So...

OLTP runs the application.

OLAP analyses the data.

Now one interesting question—

MongoDB... OLTP hai ya OLAP?"

2026-10-02 13:22:34 You 

i think i video he explain hindi and english mix

 ChatGPT gpt-5-6-t-mini

Yes — **the reference video uses a natural Hindi + English mix**, not pure English and not formal Hindi.

The delivery is more like how a Data Engineer would casually explain something to another person:

Day 3 — OLTP vs OLAP

“OLTP aur OLAP...

dono database hain,
but inka use-case completely different hai.

Let's take a simple example.

Suppose aap Amazon pe order place karte ho.

Order create hua...
payment hui...
stock update hua...

ye jo saari day-to-day transactions hain,
ye handle karta hai **OLTP**.

OLTP ka main focus hota hai...

fast transactions.

Matlab continuously data insert karna,
update karna, delete karna...

and that too very quickly.

But ab maan lo business ko analyse karna hai...

last one year mein

kaunsa product sabse zyada sell hua?

kis city se maximum orders aaye?

monthly revenue kitna tha?

Ab ye queries kaafi large data scan kar sakti hain.

Aur yahan use hota hai **OLAP**.

OLAP ka kaam hai

large amount of historical data ko analyse karna.

So simple language mein...

OLTP is for running the business.

Aur...

OLAP is for analysing the business.

Normally architecture kuch aisa hota hai...

Application...

then **OLTP database**...

then data pipeline...

and finally **OLAP warehouse**...

jahan se Power BI jaise tools reporting aur analytics karte hain.

Toh ek simple rule yaad rakho...

Transaction karni hai? — OLTP.

Data analyse karna hai? — OLAP.

Aur next question ye hai...

OLTP ka data OLAP tak jaata kaise hai?"

2026-10-02 13:25:38 You 

As like this format explain next topic proper

Day 4 — Data Lake vs Data Warehouse vs Lakehouse

“Abhi humne samjha ki...

OLTP mein day-to-day transactions hote hain,
aur **OLAP** mein hum us data ko analyse karte hain.

But ek question aata hai...

OLAP ke liye data rakhenge kahan?

Yahin pe aate hain...

Data Lake, Data Warehouse, aur Lakehouse.

Let's take a simple example.

Suppose ek e-commerce company hai.

Data aa raha hai...

SQL database se orders,
CRM se customer data,
APIs se product information,
aur website se JSON logs.

Ab itne saare different types ke data ko store karna hai.

Sabse pehle...

Data Lake.

Data Lake basically ek large storage hai
jahan aap raw data ko almost kisi bhi format mein store kar sakte ho.

CSV...

JSON...

Parquet...

logs...

matlab data ko pehle se perfectly structured hona zaroori nahi hai.

But problem ye hai...

raw data ko directly analyse karna messy ho sakta hai.

Ab business ko proper reporting chahiye...

sales report,
revenue analysis,
customer insights...

Yahan use hota hai **Data Warehouse**.

Warehouse mein data ko clean, structured
aur analytics ke liye properly organize kiya jata hai.

So ab tak simple hai...

Lake — store raw and flexible data.

Warehouse — serve structured analytics.

But modern data platforms mein ek aur approach hai...

Lakehouse.

Lakehouse ka idea simple hai...

**Data Lake ki flexibility...
plus Data Warehouse ki capabilities.**

Matlab aap lake jaisi storage flexibility rakh sakte ho,
but saath mein reliable tables, transactions
aur analytics bhi support kar sakte ho.

So ek simple rule yaad rakho...

Raw and flexible data? — Data Lake.

Structured business analytics? — Data Warehouse.

Dono ki capabilities ek platform mein? — Lakehouse.

Aur ab next question naturally aata hai...

Lakehouse ke andar data ko organize kaise karte hain?"

2026-10-02 13:28:25 You 

continue next topic

 ChatGPT gpt-5-6-t-mini

Day 5 — Medallion Architecture: Bronze vs Silver vs Gold

“Ab humne samjha...

Data Lake, Data Warehouse, aur Lakehouse kya hai.

But Lakehouse mein data aa gaya...

ab us data ko organize kaise karoge?

Imagine karo...

aapke paas raw data aa raha hai.

Orders...

customers...

products...

website logs...

sab mixed hai.

Agar isi raw data ke upar directly reporting start kar di...

toh problem hogi.

Data inconsistent ho sakta hai,

duplicates ho sakte hain,

aur cleaning bhi properly nahi hui hogi.

Isi problem ko solve karne ke liye

modern data platforms mein ek common pattern use hota hai...

Medallion Architecture.

Isme data ko mainly three layers mein divide karte hain...

Bronze... Silver... Gold.

Sabse pehle **Bronze layer.**

Yahan source se jo data aata hai,

mostly **raw form mein** store karte hain.

Matlab source mein jaisa data hai...

waise hi preserve karna.

No unnecessary business transformation.

Phir data jaata hai...

Silver layer mein.

Yahan hum data ko clean aur standardize karte hain.

Duplicates remove karna...
nulls handle karna...
data types correct karna...
columns standardize karna...

Basically...

raw data ko trustworthy data banana.

Aur finally aata hai...

Gold layer.

Yahan data business use-case ke liye ready hota hai.

For example...

sales summary,
customer analytics,
revenue metrics...

matlab Power BI ya kisi reporting tool ko jo data chahiye...

wo yahan milta hai.

So flow yaad rakho...

Bronze → Raw data

Silver → Clean and trusted data

Gold → Business-ready data

Ek simple line mein...

Bronze mein data preserve karo.

Silver mein data clean karo.

Gold mein business ko serve karo.

Aur ab ek important question...

Bronze aur Silver mein ye data store kis format mein hoga?

Yahin se **Parquet vs Delta Lake** interesting ho jata hai."

2026-10-02 13:32:18 You 

continue next

Day 6 — Parquet vs Delta Lake

“Abhi humne dekha...

Lakehouse mein data ko

Bronze, Silver aur Gold layers mein organize karte hain.

But ek simple question...

In layers ke andar data store kaise hoga?

Most commonly aap **Parquet** files dekhoge.

Parquet ek **file format** hai.

Matlab data ko efficiently store karne ke liye use hota hai...

columns mein data store karta hai,

isliye analytics queries ke liye kaafi efficient hai.

But ab problem dekho...

Maan lo customer ka address change ho gaya.

Ya kisi order ko update karna hai...

Parquet basically ek file format hai.

Usmein directly database jaisa...

UPDATE, DELETE, MERGE karna easy nahi hai.

Aur yahin pe aata hai...

Delta Lake.

Delta Lake Parquet ke upar additional capabilities add karta hai.

Jaise...

ACID transactions...

schema enforcement...

schema evolution...

UPDATE...

DELETE...

MERGE...

aur time travel.

For example...

agar mujhe existing customer record update karna hai...

Delta mein main **MERGE** use karke
existing record ko update kar sakta hoon
ya new record ko insert kar sakta hoon.

So simple difference samjho...

Parquet — data ka file format.

Delta Lake — reliable data tables ke liye additional layer.

Aur isi wajah se...

modern Lakehouse architecture mein
aapko Parquet-based data ke saath
Delta tables bhi bahut commonly milengi.

Bas ek line yaad rakho...

Parquet stores the data.

Delta makes that data more reliable and manageable.

Aur ab next question...

**Agar source mein sirf 10,000 records change hue hain,
toh kya hum har baar poora data dobara load karenge?"**

2026-10-02 13:34:40 You 

continue next



ChatGPT gpt-5-6-t-mini

Day 7 — Full Load vs Incremental Load

"Ek baar imagine karo...

Aapke database mein **10 crore records** hain.

Aur aaj sirf **20,000 records change hue hain.**

Ab pipeline run karte time...

kya hum saare 10 crore records dobara load karenge?

Technically kar sakte ho...

but production mein ye approach kaafi expensive ho sakti hai.

Because har run mein aap same old data ko baar-baar process kar rahe ho.

Isko bolte hain **Full Load**.

Full load ka matlab simple hai...

source ka poora data uthao...

aur target mein dobara load karo.

For example...

pehli baar table mein 10 crore records hain.

Next day bhi...

pipeline phir se poore 10 crore records process karegi.

Ab doosra approach dekho...

Incremental Load.

Ismein hum sirf wahi data process karte hain...

jo **new hai ya change hua hai**.

Agar 10 crore mein aaj sirf 20,000 records change hue...

toh ideally pipeline ko sirf unhi 20,000 records ke saath kaam karna chahiye.

Toh difference simple hai...

Full Load → process everything.

Incremental Load → process only what changed.

Full load useful ho sakta hai...

jab data small ho,
ya initial load kar rahe ho.

But large production tables ke liye...

incremental loading processing time, data movement aur cost ko kaafi reduce kar sakta hai.

Aur ab next question...

Pipeline ko pata kaise chalega ki ye 20,000 records new ya changed hain?

Yahin se aata hai...

Watermark."

[continue good](#) ChatGPT gpt-5-6-t-mini

Day 8 — Watermark-Based Incremental Load

“Abhi humne dekha...

Incremental load mein hum sirf
new ya changed records process karte hain.

But pipeline ko ye pata kaise chalega...

kaunse records last run ke baad change hue?

Suppose hamare Orders table mein ek column hai...

updated_at.

Pichli baar pipeline successfully chali thi...

aur last processed record ka timestamp tha...

10:00 AM.

Ab next run mein hum source se bol sakte hain...

mujhe sirf woh records chahiye
jinka `updated_at` **10:00 AM ke baad** hai.

So...

text

```
Last Watermark
10:00 AM
    ↓
updated_at > 10:00 AM
    ↓
New / Changed Records
```

Bas ye timestamp...

hamara **watermark** hai.

Aur pipeline ka flow kuch aisa hoga...

source se changed records read karo...

target mein load karo...

aur jab pipeline successfully complete ho jaye...

toh latest processed timestamp ko
next watermark bana do.

For example...

pehle watermark tha **10:00 AM**.

Is run mein latest record mila **11:30 AM**.

Toh next run ka watermark hoga...

11:30 AM.

Lekin ek important point...

watermark ko **pipeline successful hone ke baad hi update karna hai**.

Agar pipeline 11:30 AM wale data ko process karte time fail ho gayi...

aur aapne watermark pehle hi 11:30 kar diya...

toh next run mein woh records skip ho sakte hain.

So simple rule...

Watermark tells your pipeline where to start from the next run.

Aur timestamp available nahi ho...

ya INSERT, UPDATE, DELETE changes properly capture karne ho...

toh phir sirf watermark enough nahi hai.

Wahan aata hai...

CDC – Change Data Capture.”

2026-10-02 15:34:43

You



continue proper good cover



ChatGPT gpt-5-6-t-mini

Day 9 – CDC (Change Data Capture)

“Abhi humne dekha...

incremental load ke liye

hum watermark use kar sakte hain.

But ek situation socho...

source table mein **updated_at** column hi nahi hai.

Aur humein ye bhi pata karna hai ki...

kaunsa record **insert** hua,

kaunsa **update** hua,

aur kaunsa **delete** hua.

Ab sirf watermark se ye easily track nahi kar paayenge.

Yahin pe aata hai...

CDC – Change Data Capture.

CDC ka simple meaning hai...

database mein jo actual changes ho rahe hain,
un changes ko capture karna.

For example...

pehle customer ka city tha **Pune**.

Customer ne address change kiya...

ab city ho gayi **Mumbai**.

CDC is change ko capture karega...

Customer 101 – UPDATE

Similarly,

new customer create hua...

INSERT

Aur koi record delete hua...

DELETE

Toh CDC basically humein changes ka record deta hai.

text

```
INSERT
UPDATE
DELETE
↓
CDC
↓
Data Pipeline
↓
Target
```

Ab watermark aur CDC ko compare karo...

Watermark bolta hai —
‘is point ke baad ka data le aao.’

CDC bolta hai —
‘ye exact changes hue hain.’

Aur ye difference production pipelines mein kaafi important hai.

Especially jab source mein reliable timestamp nahi ho...

ya humein **update aur delete bhi properly capture karne hain.**

So simple rule yaad rakho...

Watermark tracks a position.

CDC tracks the changes.

Aur ab ek aur real-world problem hai...

CDC se changes mil gaye...

but agar pipeline fail hone ke baad same changes dobara process ho gaye toh?

Duplicates kaise prevent karenge?

Uske liye samajhna padega...

Idempotency.”

2026-10-02 15:49:03 You 

continue next with proper maneer with good script

Day 10 — Idempotency, Duplicate Data & Late-Arriving Data

“Abhi tak humne dekha...

incremental load kaise karte hain,
watermark kaise use karte hain,
aur CDC se changes kaise capture karte hain.

But production mein ek aur problem aati hai...

what if your pipeline fails... and you have to run it again?

Suppose CDC se humein **10,000 changed records** mile.

Pipeline ne 7,000 records load kar diye...

aur beech mein failure aa gaya.

Ab pipeline ko retry karna padega.

Retry mein wahi records phir se aa sakte hain.

Agar humne kuch protection nahi rakha...

toh same record target mein do baar aa sakta hai.

Aur yahin pe aata hai...

Idempotency.

Simple language mein...

pipeline ko ek hi data dobara process karo...

toh final result change nahi hona chahiye.

For example...

pehli run mein Order 101 load hua.

Retry hua...

but Order 101 duplicate nahi hona chahiye.

Iske liye hum commonly use karte hain...

unique keys, deduplication, MERGE ya upsert logic.

So...

text

Same Input



```
Run 1
  ↓
Run 2
  ↓
Same Final Result
```

But duplicate sirf retry ki wajah se nahi aate.

Kabhi-kabhi data **late** bhi arrive karta hai.

Suppose order actually **10:05 AM** pe create hua...

but source ne woh record **10:30 AM** pe bheja.

Agar hum sirf current processing window dekhen...

toh woh record miss ho sakta hai.

Isliye production pipelines mein humein...

late-arriving data ka bhi dhyan rakhna padta hai.

Aur real systems mein...

duplicates, retries, late data...

ye sab normal situations hain.

So reliable pipeline ka matlab sirf data load kar dena nahi hai.

Pipeline ko ye bhi handle karna chahiye...

same data dobara aaye,

data late aaye,

ya pipeline retry ho...

result phir bhi correct rehna chahiye.

Bas ek line yaad rakho...

A good pipeline should be safe to run again.

Aur ab jab humein reliable incremental processing samajh aa gaya...

next question hai...

agar mujhe ek nahi, hundreds of tables ko automatically process karna ho, toh kya har table ke liye alag pipeline banaunga?"



ChatGPT gpt-5-6-t-mini

Day 11 — Metadata-Driven Pipelines

“Abhi tak humne dekha...

incremental load kaise karte hain,
watermark aur CDC kaise use karte hain,
aur pipeline retry hone par duplicates kaise avoid karte hain.

But ab ek practical problem socho...

Aapki company mein sirf ek table nahi hai.

Maan lo **200 tables** hain.

Customers...

Orders...

Products...

Payments...

Transactions...

Ab kya hum 200 tables ke liye
200 alag pipelines banayenge?

Obviously... nahi.

Because kal agar ek aur table add ho gaya...
toh phir ek aur pipeline?

Isse maintenance bahut difficult ho jayegi.

Yahin pe aata hai...

Metadata-Driven Pipeline.

Simple language mein...

pipeline ke andar table names hardcode karne ke instead,
hum ek **metadata table** maintain karte hain.

Is metadata mein hum information rakh sakte hain...

source table ka naam,
target table ka naam,
load type...

full ya incremental,
watermark column...
aur source connection.

For example...

text	
Metadata	

Customers	Incremental
Orders	Incremental
Products	Full
Payments	CDC

Ab ek generic pipeline is metadata ko read karegi...

aur uske basis par decide karegi...

kaunsi table process karni hai,
kaise process karni hai,
aur kahan load karni hai.

So instead of this...

text	
200 Tables	
↓	
200 Pipelines ❌	

hum kar sakte hain...

text	
Metadata Table	
↓	
Generic Pipeline	
↓	
ForEach	
↓	
200 Tables	

Aur sabse useful part...

kal ek new table add karni hai?

Pipeline ko rewrite karne ki zarurat nahi.

Bas metadata mein configuration add karo...

and the same pipeline can process it.

Isi approach ko hum bolte hain...

Metadata-Driven Pipeline.

Simple rule yaad rakho...

More tables doesn't mean more pipelines.

Configuration badhao... pipeline nahi.

Aur ab hum data ko dynamically move kar sakte hain...

but ek question abhi bhi hai—

ye pipeline source se connect kaise karegi, data ko identify kaise karegi, aur execution ko control kaise karegi?

Yahin se samajhna important hai...

Azure Data Factory ke core components."

2026-10-02 16:06:18 You 

i want to explain ADF in seperate series so cover DE concept

 ChatGPT gpt-5-6-t-mini

Yes. **ADF should be completely separate.**

This 20-video series should teach **core Data Engineering concepts**, independent of any specific tool like ADF, Databricks, Fabric, or Snowflake.

20-Day Core Data Engineering Series

Day 1 — What is Data Engineering?

Data lifecycle: source → ingestion → storage → processing → modeling → serving → monitoring.

Day 2 — OLTP vs OLAP

Operational systems vs analytical systems, with a real e-commerce example.

Day 3 — ETL vs ELT

How data moves from source systems into analytical platforms.

Day 4 — Data Lake vs Data Warehouse vs Lakehouse

One connected explanation: raw storage vs structured analytics vs hybrid architecture.

Day 5 – Batch Processing vs Streaming

When to process data in batches and when real-time processing is needed.

Day 6 – Data Ingestion

Database, API, files, SaaS, logs, events → ingestion layer → data platform.

Day 7 – Full Load vs Incremental Load

Why processing everything repeatedly is inefficient and when incremental loading makes sense.

Day 8 – Watermarking

How a pipeline knows where the previous load stopped and where the next load should start.

Day 9 – CDC (Change Data Capture)

INSERT, UPDATE, DELETE capture and **Watermark vs CDC**.

Day 10 – Idempotency + Duplicate + Late-Arriving Data

Retries, duplicate records, late events, deduplication, upsert.

Day 11 – Data Modeling Basics

Entities, relationships, primary key, foreign key, normalization vs denormalization.

Day 12 – Fact Table vs Dimension Table + Star Schema

What happened vs describing what happened, then connect both into a star schema.

Day 13 – Star Schema vs Snowflake Schema + SCD Type 1 & 2

Schema design plus handling changing dimension history.

Day 14 – Medallion Architecture

Bronze → Silver → Gold and why each layer exists.

Day 15 – Parquet vs Delta Lake

File format vs reliable table/storage layer; ACID, MERGE, UPDATE, DELETE, schema evolution, time travel.

Day 16 – Apache Spark Fundamentals

Why distributed processing is needed, Spark architecture, DataFrame, transformations, actions, lazy evaluation.

Day 17 – Spark Partitions + Shuffle

How Spark distributes data, why shuffle happens, and why shuffle is expensive.

Day 18 – Spark Performance: Repartition vs Coalesce + Data Skew + Salting

Partition control, skewed data, and practical ways to distribute workload.

Day 19 – Spark Joins + Broadcast Join + AQE

Large-large joins, small-dimension broadcast, adaptive execution, and when each technique helps.

Day 20 – Data Quality + Schema Evolution + Monitoring + Complete DE Architecture

Bring everything together into one production Data Engineering system.

The connection

text

```
graph TD
    A[DE Fundamentals] --> B[OLTP / OLAP]
    B --> C[ETL / ELT]
    C --> D[Lake / Warehouse / Lakehouse]
    D --> E[Batch / Streaming]
    E --> F[Ingestion]
    F --> G[Full / Incremental]
    G --> H[Watermark / CDC]
    H --> I[Idempotency / Late Data]
    I --> J[Data Modeling]
    J --> K[Fact / Dimension / Star]
    K --> L[SCD]
    L --> M[Medallion]
    M --> N[Parquet / Delta]
    N --> O[Spark]
    O --> P[Partition / Shuffle]
    P --> Q[Skew / Repartition / Coalesce]
    Q --> R[Joins / Broadcast / AQE]
    R --> S[Quality / Monitoring]
    S --> T[END-TO-END DATA ENGINEERING]
```

Then your **ADF series starts separately** and teaches how to implement the ingestion/orchestration concepts practically using ADF. This keeps the core DE series tool-independent and avoids mixing “**what Data Engineering is**” with “**how ADF implements it.**”

continue pending concept proper and i thought after ADF i can cover [spark and pyspark]. also so cover instead of this pending concept and one first 3 days also missing .



Yes — now the structure is clear.

Core DE Series = only Data Engineering concepts.

ADF = separate series.

After ADF = Spark + PySpark separate series.

So I would remove ADF, Spark and PySpark from this 20-day core series and use those slots for the **remaining important DE fundamentals.**

Correct 20-Day Core Data Engineering Series

Day 1 — What is Data Engineering?

Data Engineer role, data lifecycle, source → ingestion → storage → processing → serving.

Day 2 — OLTP vs OLAP

Operational vs analytical systems, transactions vs analytics, real-world architecture.

Day 3 — ETL vs ELT

Extract, Transform, Load vs Extract, Load, Transform, and where each fits.

Day 4 — Data Lake vs Data Warehouse vs Lakehouse

All three together: purpose, architecture, differences, and when to use which.

Day 5 — Batch Processing vs Streaming

Batch, micro-batch, real-time/streaming, latency and use cases.

Day 6 — Data Ingestion

Sources, ingestion layer, structured/semi-structured data, push vs pull at a basic level.

Day 7 — Full Load vs Incremental Load

Why full load becomes expensive and why incremental processing is preferred for changing data.

Day 8 — Watermarking

Timestamp/ID watermark, last successful value, next-run logic, and failure scenario.

Day 9 — CDC (Change Data Capture)

INSERT, UPDATE, DELETE, CDC vs watermark, and where CDC fits.

Day 10 — Idempotency + Duplicates + Late Data

Retries, duplicate processing, late-arriving records, deduplication, upsert concept.

Day 11 — Data Modeling Basics

Entities, relationships, primary key, foreign key, normalization vs denormalization.

Day 12 — Fact Table vs Dimension Table + Star Schema

Facts, dimensions, measures, attributes, relationships, and star schema together.

Day 13 — Snowflake Schema + SCD Type 1 & Type 2

Normalized dimensions, when snowflake appears, overwriting vs preserving history.

Day 14 — Data Warehouse Design + Grain

What is grain, why defining grain matters, one row represents what?, additive/semi-additive/non-additive measures.

Day 15 — Medallion Architecture

Bronze → Silver → Gold, raw → cleaned → business-ready, and how it connects with the Lakehouse.

Day 16 — File Formats: CSV vs JSON vs Parquet vs Avro

Why different formats exist, row vs column orientation, schema, compression, analytics use cases.

Day 17 — Partitioning + Data Distribution

Why large datasets are divided, partition pruning concept, folder/data organization, and common mistakes.

Day 18 — Schema Evolution + Schema Drift + Data Contracts

Expected vs unexpected schema changes, pipeline breakage, producer-consumer agreement.

Day 19 — Data Quality + Data Governance

Nulls, duplicates, completeness, accuracy, freshness, validation, lineage, catalog, access control.

Day 20 — Data Reliability + End-to-End DE Architecture

Retries, idempotency, monitoring, alerting, SLAs/SLOs at a basic level, and connect the complete architecture.

The flow now becomes

text

```
Day 1 → What is Data Engineering
Day 2 → OLTP vs OLAP
Day 3 → ETL vs ELT
      ↓
Day 4 → Lake / Warehouse / Lakehouse
Day 5 → Batch / Streaming
Day 6 → Ingestion
      ↓
Day 7 → Full / Incremental
```

Day 8 → Watermark
Day 9 → CDC
Day 10 → Idempotency / Late / Duplicate
↓
Day 11 → Data Modeling
Day 12 → Fact / Dimension / Star
Day 13 → Snowflake / SCD
Day 14 → Warehouse Design / Grain
↓
Day 15 → Medallion
Day 16 → File Formats
Day 17 → Partitioning
Day 18 → Schema Evolution / Drift
Day 19 → Quality / Governance
Day 20 → Reliability / Complete Architecture

Then the learning path continues separately:

text

CORE DE CONCEPTS
↓
20-Day DE Series
↓
ADF Series
↓
Spark + PySpark Series
↓
Databricks / Fabric / Cloud Implementation

This is much cleaner because the audience first understands **Data Engineering itself**, then learns **ADF as an orchestration tool**, and only after that moves into **Spark/PySpark for distributed processing**.

2026-10-02 16:08:26 You 

continue pending

 ChatGPT gpt-5-6-t-mini

Day 11 — Data Modeling: Normalization vs Denormalization

“Abhi tak humne data ko store karna samjha...
kaise load karna hai,

incremental data kya hota hai,
watermark aur CDC kaise kaam karte hain.

But ab ek important question hai...

Data warehouse ke andar data ko actually design kaise karoge?

Suppose ek e-commerce company hai.

Aapke paas customer hai...

product hai...

aur orders hain.

Ab imagine karo ek customer ne 50 orders place kiye.

Agar customer ka name, email, city...

har order ke saath repeat kar diya...

toh same information baar-baar store hogi.

Ye hai **data redundancy**.

Aur redundancy ko reduce karne ke liye

ek approach hai...

Normalization.

Ismein hum data ko logically different tables mein divide karte hain.

For example...

Customer ki information ek table mein...

Product ki information doosre table mein...

aur Orders alag table mein.

Phir primary key aur foreign key se inko connect kar dete hain.

text

Customer



Customer_ID

Orders



Customer_ID

Isse duplicate data kam hota hai

aur data consistency better maintain karna easy hota hai.

But analytics mein ek different problem aa sakti hai.

Agar ek report ke liye mujhe
Customer... Order... Product...
multiple tables join karni pad rahi hain...

toh query complicated ho sakti hai.

Yahin pe hum **Denormalization** use kar sakte hain.

Matlab related data ko analytics ke liye
thoda combine ya duplicate karna...

taaki queries simpler ho sakein.

So simple way mein...

Normalization ka focus hai — redundancy kam karna.

Aur...

Denormalization ka focus hai — analytics ko simpler banana.

But yahan ek important thing hai...

Normalise kitna karna hai?

Denormalise kitna karna hai?

Ye depend karta hai system ka use-case kya hai.

Transaction system mein normalization common hai...

aur analytical systems mein denormalized models kaafi common milte hain.

Aur isi se ek important question aata hai...

Analytics ke liye kaunsi tables actually important hoti hain?

Yahin se samajhte hain...

Fact Table aur Dimension Table."

2026-10-02 16:09:05 You 

continue next with good script

Day 12 — Fact Table vs Dimension Table + Star Schema

“Abhi humne data modeling mein dekha...

normalization se data ko logically separate kar sakte hain,
aur denormalization se analytics ko simpler bana sakte hain.

But ab data warehouse design mein
sabse important question hai...

Fact table kya hoti hai...

aur Dimension table kya hoti hai?

Ek simple e-commerce example lete hain.

Suppose ek customer ne ek product buy kiya.

Ab is transaction ke baare mein
humein kya jaana hai?

Kitne products buy kiye?

Kitne paise pay kiye?

Discount kitna tha?

Ye jo numbers hain...
ye business event ko **measure** karte hain.

Aur aise data ko hum usually
Fact Table mein rakhte hain.

So...

Fact table tells us — what happened.

For example...

text

```
Order_ID
Customer_ID
Product_ID
Date_ID
Quantity
Sales_Amount
Discount
```

Ab doosri side dekho.

Humein ye bhi know karna hai...

Customer kaun tha?

Product kaunsa tha?

Order kis date ko hua?

Customer kis city se hai?

Product kis category ka hai?

Ye descriptive information hoti hai...

aur isko hum **Dimension Tables** mein rakhte hain.

For example...

text

Customer Dimension

Customer_ID

Name

City

Country

text

Product Dimension

Product_ID

Product_Name

Category

Brand

Toh simple difference...

Fact = measurements.

Dimension = description.

Ab in dono ko connect kaise karenge?

Fact table ke andar

Customer_ID, Product_ID, Date_ID jaise keys honggi...

aur ye respective dimension tables ko refer karenge.

Aur jab ek central fact table ke around

multiple dimension tables arrange karte hain...

toh jo design banta hai usko bolte hain...

Star Schema.

Something like this...

text



Fact table center mein...

aur dimensions uske around.

Isliye naam hai...

Star Schema.

Ab ek quick test.

Agar question hai...

‘Total sales kitni hui?’

We look at the **Fact**.

Aur agar question hai...

‘Mumbai se kitni sales hui?’

Mumbai ka information kahan hoga?

Dimension mein.

So yaad rakho...

Fact tells you what happened.

Dimension tells you how you want to describe or analyse what happened.

Aur jab fact ko dimensions ke saath center mein connect karte ho...

that’s a Star Schema.

Ab naturally next question aata hai...

Agar Product ke andar Category hai,

Category ke andar Department hai...

toh kya har dimension ko ek hi table mein rakhna zaroori hai?

Yahin se interesting hota hai...

continue in good simple cover all good manner



ChatGPT gpt-5-6-t-mini

Day 13 – Star Schema vs Snowflake Schema + SCD Type 1 & Type 2

“Abhi humne dekha...

Fact table center mein hoti hai,
aur uske around Customer, Product, Date jaise dimensions hoti hain.

Isko hum bolte hain...

Star Schema.

But ab ek question.

Agar Product dimension mein
Product ke saath Category bhi hai,
aur Category ke saath Department bhi hai...

toh kya ye saari information ek hi table mein rakhni chahiye?

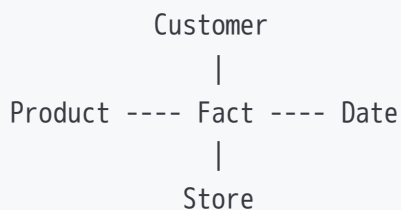
Ya phir isko alag-alag tables mein divide kar sakte hain?

Yahin pe aata hai...

Snowflake Schema.

Star Schema mein dekho...

text



Dimensions directly fact table se connected hain.

But Snowflake mein dimension ko bhi further break kar sakte hain.

For example...

text

Fact

|

Product

|

Category

|

Department

Matlab Product table mein sirf category name repeat karne ke instead...

Product ko Category se connect kar diya,
aur Category ko Department se.

Isliye simple difference...

Star Schema – simple and denormalized.

Snowflake Schema – more normalized and more tables.

Star mein queries generally simple hoti hain...

Snowflake mein relationships zyada ho sakte hain,
toh joins bhi zyada ho sakte hain.

But ab ek aur real-world problem dekho.

Customer ne profile create ki...

Customer City = **Pune**.

Six months later...

customer Mumbai shift ho gaya.

Ab question hai...

Kya hum Pune ko replace karke Mumbai kar dein?

Agar old value ki history important nahi hai...

toh hum use kar sakte hain...

SCD Type 1.

Matlab simple...

text

Pune
↓
Mumbai

Old value overwrite ho gayi.

But suppose business poochta hai...

“Jab customer ne Pune se order kiya tha,
tab uska city kya tha?”

Ab agar humne Pune overwrite kar diya...

history lost ho chuki hai.

So yahan use karte hain...

SCD Type 2.

Type 2 mein old record delete nahi karte.

Hum uska ek historical version maintain karte hain.

For example...

text

Customer 101	Pune	Old
Customer 101	Mumbai	Current

Aur usually start date, end date,
ya current flag jaise columns bhi rakhte hain.

So...

Type 1 → old value overwrite.

Type 2 → old value preserve and new version create.

Bas ek simple rule yaad rakho...

History nahi chahiye? → Type 1.

History chahiye? → Type 2.

Aur ye concept data warehouse mein bahut important hai...

because business ko sirf current data nahi,
kabhi-kabhi **historical state** bhi analyse karni hoti hai.”

[continue next](#)

Day 14 — Data Warehouse Design + Grain

“Abhi tak humne dekha...

Fact table kya hoti hai,
Dimension table kya hoti hai,
Star Schema kaise banta hai,
aur SCD Type 1 aur Type 2 kya hota hai.

But ek cheez hai...

jo table design karne se pehle hi decide karni padti hai.

Fact table mein ek row actually represent kya karegi?

Isko bolte hain...

Grain.

Grain matlab...

‘One row represents what?’

Let’s take a simple example.

Suppose hamari e-commerce company mein
ek customer ek order place karta hai.

Ab agar hum decide karein ki...

one row = one order

toh ye hamare fact table ka grain hai.

But same order mein customer ne
3 different products buy kiye.

Ab agar business ko product-level sales analyse karni hai...

toh one row = one order enough nahi hoga.

Humein decide karna padega...

one row = one order line.

For example...

text

Order 101

|—— Mobile

|—— Cover

|—— Charger

Toh fact table mein
3 rows ho sakti hain.

Aur yahi decision bahut important hai.

Because agar grain clear nahi hai...

toh quantities, revenue, counts...

sab wrong calculate ho sakte hain.

For example...

agar ek order mein 3 products hain
aur aap order aur order-line data ko mix kar do...

toh revenue accidentally double count bhi ho sakta hai.

So data warehouse design mein
sabse pehle ye clear karo...

one row represents what?

Ab grain decide hone ke baad
hum facts bhi identify kar sakte hain.

Jaise...

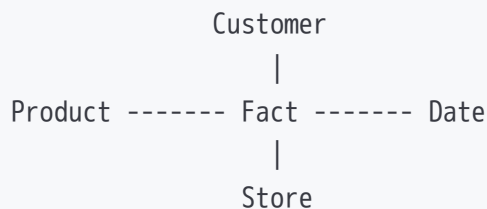
Quantity,
Sales Amount,
Discount.

Aur dimensions honggi...

Customer,
Product,
Date,
Store.

So overall design kuch aisa ho sakta hai...

text



Fact ka grain...

one row = one order line.

Ab aapko pata hai
exactly kya store karna hai,
kis level pe store karna hai,
aur dimensions ke saath kaise analyse karna hai.

Isliye ek simple rule yaad rakho...

Before designing a fact table, define the grain first.

Because...

if the grain is wrong, the whole analysis can be wrong."

2026-10-02 16:24:23 You 

continue next as per list if anything pending

 ChatGPT gpt-5-6-t-mini

Day 15 — Medallion Architecture: Bronze, Silver & Gold

"Abhi humne data warehouse design karna samjha...

Fact table, Dimension table,
Star Schema, SCD...

But modern Data Lake ya Lakehouse mein ek aur important question hai...

Raw data ko directly business ko de dein?

Obviously nahi.

Suppose source se data aa raha hai...

customers,
orders,

payments...

aur source mein data messy bhi ho sakta hai.

Kuch duplicate records hain...

kuch null values hain...

different systems mein date format bhi different ho sakta hai.

Toh hum is data ko directly reporting ke liye use nahi karte.

Hum data ko different layers mein process karte hain.

Is approach ko bolte hain...

Medallion Architecture.

Ismein mainly teen layers hoti hain...

Bronze, Silver, and Gold.

Sabse pehle...

Bronze Layer.

Yahan source se jo data aata hai...

usse mostly raw form mein store karte hain.

Matlab source mein jaisa data hai...

ussi form mein preserve karna.

text

Source



BRONZE



Raw Data

Ab Bronze ka data directly reporting ke liye use karna ideal nahi hai.

So next layer...

Silver Layer.

Yahan actual data transformation hoti hai.

Duplicates remove karna...

null values handle karna...

data types correct karna...

columns standardize karna...

aur different sources ka data properly clean karna.

So...

text

BRONZE

Raw



SILVER

Clean + Standardized

Ab Silver mein data trustworthy ho gaya.

But business ko raw customer table ya cleaned transaction table nahi chahiye.

Unko chahiye...

Revenue by month.

Sales by product.

Top customers.

Aur aise business-ready datasets hum rakhte hain...

Gold Layer mein.

text

SILVER

Clean Data



GOLD

Business-ready Data

For example...

Silver mein hundreds of millions of order records ho sakte hain.

Gold mein hum usi data se bana sakte hain...

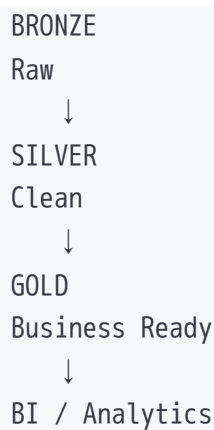
Daily Sales Summary.

So complete flow dekho...

text

SOURCE





Ek simple way mein yaad rakho...

Bronze mein data preserve karo.

Silver mein data trustworthy banao.

Gold mein business ko serve karo.

Aur sabse important...

Bronze, Silver, Gold alag-alag random folders nahi hain.

Ye actually ek **data processing strategy** hai...

jahan data step-by-step raw se business-ready banta hai.”

2026-10-02 16:24:45 You 

completed already medillion

 ChatGPT gpt-5-6-t-mini

Day 16 — Parquet vs Delta Lake

“Abhi humne Medallion Architecture mein dekha...

Bronze mein raw data aata hai,
Silver mein clean hota hai,
aur Gold mein business-ready data milta hai.

But ek simple question...

Ye data actually store kis format mein hota hai?

Suppose Bronze layer mein
hamare paas millions of orders hain.

Hum is data ko files mein store kar sakte hain...

aur yahan ek common format hai...

Parquet.

Parquet ek **file format** hai.

Iska ek major benefit hai...

ye data ko **column-wise** store karta hai.

For example, agar table mein 50 columns hain...

aur query ko sirf 3 columns chahiye...

toh unnecessary columns ko read karne ki zarurat kam ho sakti hai.

Isliye analytical workloads mein

Parquet kaafi useful hota hai.

But ab real problem dekho...

Agar customer ka city change ho gaya...

ya kisi order ko update karna hai...

ya existing records ko new records ke saath merge karna hai...

toh simple file format se

ye database jaisa behaviour easily manage nahi hota.

Aur yahin pe aata hai...

Delta Lake.

Delta Lake basically Parquet files ke upar additional capabilities provide karta hai.

For example...

aap **ACID transactions** use kar sakte ho...

schema ko control kar sakte ho...

existing records ko **UPDATE** kar sakte ho...

records **DELETE** kar sakte ho...

aur sabse important...

MERGE kar sakte ho.

Suppose source se ek record aaya...

Order ID 101 already target mein hai.

Toh MERGE logic decide kar sakta hai...

existing hai toh **UPDATE** karo...

nahi hai toh **INSERT** karo.

Aur Delta ka ek aur useful feature hai...

Time Travel.

Matlab previous versions of data ko bhi access kar sakte ho.

So simple difference samjho...

Parquet tells you how the data is stored.

Delta Lake gives you reliable table capabilities on top of that storage.

Isliye modern Lakehouse mein
aapko bahut baar ye combination dikhega...

Parquet files underneath...

Delta tables on top.

Aur ab next question...

Humne data ko Lakehouse mein store karna bhi samajh liya...

lekin jab data bahut bada ho jaye...

itna data process kaise karenge?"

2026-10-02 16:25:01 You 

this alos completed

 ChatGPT gpt-5-6-t-mini

Day 17 — Data Partitioning

"Data Lake ya Lakehouse mein...

maan lo aapke paas **10 crore records** hain.

Ab har query ke time...

agar poora 10 crore data read karna pade...

toh obviously time lagega.

Toh question hai...

kya hum data ko logically chhote parts mein divide kar sakte hain?

Yes.

Isko bolte hain...

Data Partitioning.

Simple example lete hain.

Suppose hamare Orders table mein

January, February, March...

poore saal ka data hai.

Instead of keeping everything together...

hum data ko month ke basis pe partition kar sakte hain.

Something like...

text

Orders

```
graph TD
    Orders[Orders] --- P1[2026-01]
    Orders --- P2[2026-02]
    Orders --- P3[2026-03]
    Orders --- P4[2026-04]
```

Ab maan lo business ko sirf...

March 2026 ki sales chahiye.

Toh system ko har record read karne ki zarurat nahi honi chahiye.

Agar query filter kar rahi hai...

month = March

toh sirf relevant partition read kiya ja sakta hai.

Is concept ko bolte hain...

Partition Pruning.

Matlab unnecessary data ko read hi mat karo.

And this can make large-data queries much more efficient.

Lekin yahan ek common mistake bhi hoti hai...

har column ko partition bana dena.

Suppose hum Customer ID se partition kar dein...

aur million customers hain.

Toh suddenly...

bahut saare tiny partitions create ho jayenge.

Aur ye khud ek problem ban sakti hai.

So partition choose karte time dekho...

kya ye column commonly filter hota hai?

kya partitions ka size reasonable rahega?

kya bahut saare small files toh nahi ban rahe?

For analytics, date-based partitioning bahut common hai...

jaise year... month... day...

because queries mein date filters frequently aate hain.

So simple rule yaad rakho...

Partitioning ka goal data ko bas divide karna nahi hai.

Goal hai unnecessary data reading ko reduce karna.

Aur jab partitioning ki baat hoti hai...

ek aur important question aata hai—

data ko partition kis column par karna chahiye?

Aur yahin se samajhna important ho jata hai...

How to choose the right partition key.”

2026-10-02 16:28:51

You 

continue next



ChatGPT gpt-5-6-t-mini

Day 18 — Schema Evolution vs Schema Drift + Data Contracts



“Abhi humne data partitioning samjha...

ki large data ko properly organize karke
unnecessary data reading reduce kar sakte hain.

But production mein ek aur problem bahut common hai...

source ka structure hi change ho gaya.

Suppose aapki pipeline roz ek customer file read karti hai.

Kal tak columns the...

customer_id

name

city

Pipeline perfectly chal rahi thi.

But aaj source team ne ek new column add kar diya...

email .

Ab question hai...

pipeline break hogi... ya nahi?

Ye depend karta hai ki change expected tha ya unexpected.

Agar system mein pehle se decide tha
ki future mein new columns add ho sakte hain...

toh isse bolte hain...

Schema Evolution.

Matlab schema time ke saath change ho raha hai...

but change expected hai
aur system usko handle karne ke liye ready hai.

For example...

text

Yesterday

id | name | city

Today

id | name | city | email

But ab second situation dekho.

Source ne bina bataye...

`customer_id` ka naam change kar diya.

Ya data type INT se STRING kar diya.

Ya `city` column remove kar diya.

Aur pipeline ko kuch pata hi nahi tha.

Ye hai...

Schema Drift.

Matlab source schema mein unexpected change aa gaya...

aur downstream pipeline ko surprise mil gaya.

Aur isi wajah se production pipeline crash bhi ho sakti hai.

So difference simple hai...

Schema Evolution → expected change.

Schema Drift → unexpected change.

But ab question...

“Source team ko kaise pata hoga
ki downstream system exactly kya expect karta hai?”

Yahin pe aata hai...

Data Contract.

Data Contract basically ek agreement hai...

source kya data bhejega,
column ka naam kya hoga,
data type kya hoga,
required field kaunsi hai,
aur expected format kya hoga.

For example...

text

```
customer_id → integer
name        → string
city        → string
email       → string
```

Ab producer aur consumer dono ko same expectation pata hai.

So agar koi breaking change karna hai...

toh silently production mein change karne ke instead...

properly communicate aur validate kiya ja sakta hai.

Ek simple way mein yaad rakho...

Schema Evolution = controlled change.

Schema Drift = unexpected change.

Data Contract = agreed structure between producer and consumer.

Aur ye important hai...

because Data Engineering mein problem sirf ye nahi hoti ki data pipeline ka code sahi hai ya nahi.

Kabhi-kabhi...

source hi aapki expectation ke according data nahi bhej raha hota."

2026-10-02 16:32:50

You



continue next



ChatGPT gpt-5-6-t-mini

Day 19 — Data Quality vs Data Governance

"Ab ek important problem samjho...

Pipeline successfully run ho gayi.

Koi error nahi aaya...

status bhi **Success** hai.

But...

kya iska matlab data bhi correct hai?

Not necessarily.

Suppose kal 10 lakh orders aane chahiye the...



but aaj sirf 8 lakh records aaye.

Pipeline perfectly run ho sakti hai...

but business ke liye data already wrong hai.

Yahin pe aata hai...

Data Quality.

Data quality ka matlab simple hai...

jo data aaya hai...

kya woh complete hai, correct hai, consistent hai, aur expected time pe aaya hai?

For example...

Customer ID null toh nahi?

Same order do baar toh nahi aaya?

Sales amount negative toh nahi hai?

Expected number of records aaye ya nahi?

Aur data fresh hai ya purana?

These are all **data quality checks**.

Ek simple example dekho...

text

Expected Orders
1,000,000

Received
800,000

Pipeline → SUCCESS
Data → ❌

So yaad rakho...

Pipeline success does not mean data success.

Ab ek doosri problem.

Maan lo data correct hai...

but question hai...

data access kaun kar sakta hai?

Customer ka sensitive data
har employee ko access nahi milna chahiye.

Kisne data create kiya?

Kisne modify kiya?

Data kahan se aaya?

Aur business ko exactly kaunsa dataset use karna chahiye?

Yahan aata hai...

Data Governance.

Governance ka focus hota hai...

data ko properly **manage, protect aur control** karna.

Ismein concepts aate hain...

access control...

data ownership...

data lineage...

catalog...

aur sensitive data ko protect karna.

For example...

text

Data Source



Data Pipeline



Warehouse



Power BI

- ↘ Lineage
- ↘ Access Control
- ↘ Ownership

So difference simple hai...

Data Quality asks – “Is the data right?”

Data Governance asks – “Is the data properly managed and controlled?”

Aur production Data Engineering mein...
dono important hain.
Because business ko sirf data chahiye nahi...
trusted data chahiye.”

2026-10-02 16:36:47

You 

continue next



ChatGPT gpt-5-6-t-mini

Day 20 — Data Reliability, Monitoring & End-to-End Data Engineering

“Ab ek last important thing samjho...

Aapki pipeline data ko sahi process kar rahi hai...

data quality checks bhi hain...

but **production mein pipeline fail ho gayi** toh?

Suppose pipeline raat ko 2 baje run hoti hai.

Database se data aana tha...

but source temporarily unavailable tha.

Pipeline fail ho gayi.

Ab Data Engineer ko kya karna hai?

Yahan se aata hai...

Data Reliability.

Sabse pehle...

failure ko properly detect karna hai.

Phir logs dekhne hain...

actual error kya hai...

aur agar failure temporary hai...

toh pipeline ko **retry** karna chahiye.

But retry karte time ek problem yaad hai?

Same data dobara process ho sakta hai.

Isliye jo concept humne pehle padha tha...

Idempotency...

wo yahan kaam aata hai.

Matlab pipeline ko retry karo...

but final data duplicate ya incorrect nahi hona chahiye.

Aur production mein sirf retry enough nahi hai.

Humein **monitoring** bhi chahiye.

For example...

pipeline kitne time mein complete hui?

Kitne records process hue?

Koi unusual drop toh nahi hua?

Aur agar pipeline fail hui...

toh engineer ko automatically alert milna chahiye.

So basic flow dekho...

text

```
Pipeline
↓
Logs + Metrics
↓
Failure / Anomaly
↓
Alert
↓
Engineer
↓
Retry / Fix
```

Ab ek baar poori Data Engineering journey ko connect karte hain.

Data sabse pehle aata hai...

OLTP systems, APIs, files ya events se.

Phir data ko ingest karte hain...

aur Data Lake ya Lakehouse mein store kar sakte hain.

Raw data...

Bronze.

Clean aur standardized data...

Silver.

Business-ready data...

Gold.

Uske baad hum data ko properly model kar sakte hain...

Fact tables, Dimension tables, Star Schema...

aur historical changes ke liye...

SCD Type 2.

Large data ho toh distributed processing use kar sakte hain...

aur uske saath partitioning, joins aur performance ka bhi dhyan rakhna padta hai.

Then data quality checks...

schema validation...

monitoring...

security...

and finally...

business ko trusted data serve karna hai.

So complete picture kuch aisa hai...

text

Sources



Ingestion



Data Lake / Lakehouse



Bronze



Silver



Gold



Aur iske across...

Incremental Load...

CDC...

Data Quality...

Reliability...

Monitoring...

sab continuously kaam karte hain.

So Data Engineering sirf data ko ek jagah se doosri jagah move karna nahi hai.

Real job is to build a data system that is scalable, reliable, and trustworthy."

2026-10-02 16:41:39 You 

is the all concept complete i think first 3 day concept not completed



ChatGPT gpt-5-6-t-mini

Yes, you caught it correctly. **The first 3 core DE concepts are not properly completed as separate videos yet.**

We should keep the final 20-day sequence clean:

1. **What is Data Engineering?** — pending
2. **OLTP vs OLAP** — pending as a proper standalone script
3. **ETL vs ELT** — pending as a proper standalone script
4. **Data Lake vs Data Warehouse vs Lakehouse** — completed
5. **Medallion Architecture** — completed
6. **Parquet vs Delta Lake** — completed
7. **Full Load vs Incremental Load** — completed
8. **Watermark-Based Incremental Load** — completed
9. **CDC** — completed
10. **Idempotency + Duplicate + Late-Arriving Data** — completed
11. **Data Modeling: Normalization vs Denormalization** — completed

- 12. **Fact Table vs Dimension Table + Star Schema** — completed
- 13. **Star vs Snowflake + SCD Type 1 & 2** — completed
- 14. **Data Warehouse Design + Grain** — completed
- 15. **Medallion Architecture** — already covered, so this duplicate should be removed from the final sequence
- 16. **Parquet vs Delta Lake** — already covered, so this duplicate should also be removed
- 17. **Data Partitioning** — completed
- 18. **Schema Evolution vs Schema Drift + Data Contracts** — completed
- 19. **Data Quality + Data Governance** — completed
- 20. **Data Reliability + Monitoring + End-to-End Architecture** — completed

So I would **not continue from Day 21**. We should first fix the sequence by replacing the duplicate Medallion/Parquet slots with two important missing core DE concepts, and then properly create **Day 1, Day 2, and Day 3 as separate scripts**.

The three starting concepts should be:

- Day 1 — What is Data Engineering?**
- Day 2 — OLTP vs OLAP**
- Day 3 — ETL vs ELT**

Then the series flows naturally into:

Day 4 — Data Lake vs Data Warehouse vs Lakehouse

That gives the viewer a proper foundation instead of jumping directly into storage architecture.

2026-10-02 16:42:06 You 

continue day 1

 ChatGPT gpt-5-6-t-mini

Day 1 — What is Data Engineering?

“Jab bhi aap Swiggy pe order karte ho...
order place hota hai...
payment hoti hai...
delivery status update hota hai...”

yeh poora process data generate karta hai.

But yeh data...

store kahan hota hai?

Kaise move hota hai?

Aur jab company ko is data se business insights nikalni hoti hain...

yeh data ready kaun karta hai?

Yahin pe aata hai...

Data Engineering.

Simple language mein...

Data Engineering ka kaam hai

data ko source se lekar...

business ke liye usable form tak pahunchana.

For example...

data aa sakta hai

database se...

API se...

CSV files se...

ya application events se.

Ab is data ko hum ingest karte hain...

phir store karte hain...

clean aur transform karte hain...

aur finally analytics ya reporting ke liye ready karte hain.

Agar simple flow dekho...

text

Source



Ingestion



Storage



Transformation



Analytics

For example...

e-commerce company mein
orders ka raw data source se aaya.

Data Engineer ensure karega ki...

data properly ingest ho...

duplicate na ho...

required transformations ho...

aur final data reporting ke liye trustworthy ho.

Aur yahan ek important difference hai...

Data Analyst mostly data ko analyse karke insights nikalta hai.

But **Data Engineer** pehle woh reliable data system build karta hai...

jiske upar analysis ho sake.

So ek line mein yaad rakho...

Data Engineer ka kaam sirf data move karna nahi hai.

Reliable, usable aur scalable data system build karna hai.

Aur ab ek important question...

Ye data aata kahan se hai,

aur source database mein actually kis type ka workload hota hai?

Wahin se samajhna important hai...

OLTP vs OLAP."

2026-10-02 16:44:26 You 

continue day 2

 ChatGPT gpt-5-6-t-mini

Day 2 — OLTP vs OLAP

"Abhi Day 1 mein humne dekha tha...

Data Engineer ka actual kaam kya hota hai.

But ab ek basic question hai...

Ye data aata kahan se hai?

Suppose aap Amazon pe ek order place karte ho.

Order create hua...

payment hui...

inventory update hui...

aur order ka status bhi update ho gaya.

Ye jo saari activities hain...

ye **day-to-day transactions** hain.

Aur inko handle karne ke liye use hota hai...

OLTP.

OLTP ka full form hai...

Online Transaction Processing.

Iska main focus hota hai...

bahut saari small transactions ko quickly handle karna.

Matlab...

insert karo...

update karo...

delete karo...

aur response quickly milna chahiye.

Ab ek second ke liye imagine karo...

isi database mein business ek query run karta hai—

“Last 2 years mein kis product ne sabse zyada revenue generate kiya?”

Ab yahan problem aa sakti hai.

Because ye ek small transaction nahi hai...

ye **large amount of historical data ka analysis** hai.

Aur isi type ke workload ke liye use hota hai...

OLAP.

OLAP ka full form hai...

Online Analytical Processing.

Yahan focus hota hai...

large data ko read karna...
multiple tables ko analyse karna...
aggregation karna...
aur business insights nikalna.

Toh difference ko ek simple example se dekho...

Aapne abhi order place kiya...

OLTP.

Aur company pooch rahi hai...

“Pichle 12 months mein total sales kitni hui?”

OLAP.

So...

OLTP is for transactions.

OLAP is for analysis.

Aur isi wajah se normally architecture kuch aisa hota hai...

Application...



OLTP Database



Data Pipeline



OLAP / Data Warehouse



BI and Analytics

Matlab...

OLTP business ko run karta hai...

aur...

OLAP business ko samajhne mein help karta hai.

Bas ek simple rule yaad rakho...

Small, frequent transactions? → OLTP.

Large, historical analysis? → OLAP.

Aur ab next question...

OLTP se data OLAP tak jaata kaise hai?

Yahin se samajhna padega...

ETL vs ELT."

2026-10-02 16:45:28

You



continue next



ChatGPT gpt-5-6-t-mini

Day 3 – ETL vs ELT

"Abhi humne dekha...

OLTP mein day-to-day transactions hote hain...

aur OLAP mein hum us data ko analyse karte hain.

But ek question yahan aata hai...

OLTP ka data OLAP tak jaata kaise hai?

Maan lo e-commerce application ka order data

OLTP database mein aa raha hai.

Ab humein ye data analytics ke liye

Data Warehouse mein le jaana hai.

Simple lag raha hai...

but ek important step hai.

Source ka data directly analytics ke liye ready nahi hota.

Kuch values missing ho sakti hain...

data format different ho sakta hai...

duplicate records ho sakte hain...

toh humein data ko prepare karna padega.

Aur yahin pe aate hain...



ETL and ELT.

Pehle samjhte hain **ETL**.

ETL means...

Extract... Transform... Load.

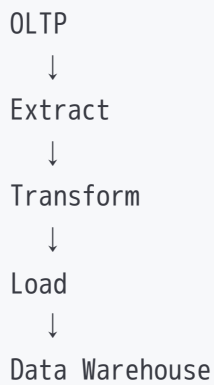
Matlab pehle source se data extract karo...

phir usko transform aur clean karo...

aur finally warehouse mein load karo.

Example dekho...

text



Ab **ELT** mein order change ho jaata hai.

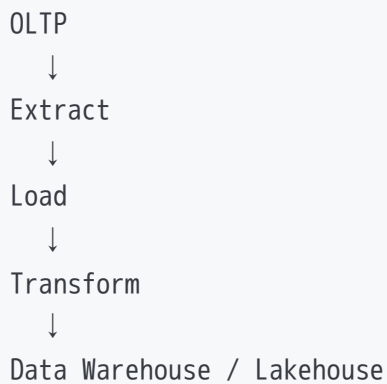
Extract... Load... Transform.

Matlab source se data nikalo...

pehle usko warehouse ya lakehouse mein load karo...

aur transformation baad mein target platform ke andar karo.

text



Ab question aata hai...

difference actually important kyun hai?

ETL mein transformation pehle ho rahi hai...

so target ko relatively clean data milta hai.

ELT mein hum raw ya near-raw data pehle store kar sakte hain...

aur phir cloud platform ki compute power use karke transform kar sakte hain.

Isi liye modern cloud data platforms mein...

ELT approach kaafi common hai.

So simple example...

Agar source se customer data aaya...

ETL mein:

Extract → clean → transform → Load.

ELT mein:

Extract → Load first → transform later.

Toh bas order yaad rakho...

ETL → Transform before Load.

ELT → Transform after Load.

Aur ek important baat...

ETL aur ELT mein koi ek universal winner nahi hai.

Aapka architecture,

data volume,

source system,

aur target platform decide karega...

ki kaunsa approach suitable hai.

So ab tak humne samjha...

OLTP data generate karta hai...

ETL ya ELT us data ko move aur prepare karta hai...

aur...

OLAP us data ko analyse karta hai.

Ab next question aur interesting hai...

Ye analytical data actually store kahan karenge?

Data Lake?

Data Warehouse?

Ya kuch aur?

Yahin se aata hai...

Data Lake vs Data Warehouse vs Lakehouse."

2026-10-02 16:46:31

You



all cover right



ChatGPT gpt-5-6-t-mini

Yes. The **Day 1–3 foundation is now properly covered and connected:**

Day 1 – What is Data Engineering?

Covers the complete data lifecycle and role of a Data Engineer.

Day 2 – OLTP vs OLAP

Covers operational vs analytical workloads, real e-commerce example, and how OLTP → data pipeline → OLAP fits together.

Day 3 – ETL vs ELT

Covers why data needs transformation, ETL flow, ELT flow, the actual difference, when each approach is used, and how it connects **OLTP → ETL/ELT → OLAP**.

Then the series naturally continues:

Day 4 → Data Lake vs Data Warehouse vs Lakehouse

So the foundation is:

text

Application



OLTP



ETL / ELT



Lake / Warehouse / Lakehouse



Nothing important from those first three concepts is intentionally skipped.

2026-10-02 17:18:24

You



all 20 days basic concept completed so now give ADF part proper -
1- build a good plan and give me how many days we cover concept
all and make sure good one format generate plan ok



ChatGPT gpt-5-6-t-mini

ADF Video Series — Complete Learning Plan

I'd make the ADF series **20 days**.

That is enough to move from **"What is ADF?"** → **"How do I build a real production pipeline?"** without mixing ADF with the Core DE series or Spark series.

The structure should be:

text

```
ADF FUNDAMENTALS
  ↓
ADF CORE COMPONENTS
  ↓
DATA MOVEMENT
  ↓
CONTROL FLOW
  ↓
DYNAMIC / REUSABLE PIPELINES
  ↓
INCREMENTAL DATA LOADING
  ↓
API / REAL-WORLD INTEGRATION
  ↓
ERROR HANDLING
  ↓
MONITORING / SECURITY
  ↓
CI/CD
  ↓
REAL END-TO-END ADF PIPELINE
```

PHASE 1 – ADF FOUNDATION

Day 1 – What is Azure Data Factory?

Cover:

- What ADF is
- Why companies use ADF
- Data integration vs data transformation
- Basic ADF architecture
- Where ADF fits in a Data Engineering system

Flow:

text

```
graph TD;
    A[Source Systems] --> B[Azure Data Factory];
    B --> C[Move / Transform / Orchestrate];
    C --> D[Data Platform];
```

Connection:

First understand the purpose of ADF before learning its components.

Day 2 – ADF Main Components

Cover the complete ADF building blocks:

- Pipeline
- Activity
- Dataset
- Linked Service
- Integration Runtime
- Trigger

Show how they work together:

text

```
Trigger
↓
Pipeline
↓
Activity
↓
Dataset
↓
Linked Service
↓
Source / Target
```

Connection:

After knowing ADF, the viewer now understands its vocabulary.

Day 3 – Linked Services

Cover:

- What is Linked Service?
- Connection information
- Source vs destination connection
- Authentication
- Azure resources
- Database connections
- Storage connections

Example:

```
text
```

```
ADF
↓
Linked Service
↓
Azure SQL
```

Explain the key idea:

Linked Service = how ADF connects to something.

Day 4 – Integration Runtime

Cover:

- What is IR?
- Azure Integration Runtime
- Self-hosted Integration Runtime
- Managed VNet IR
- When each is used
- Public vs private connectivity

Example:

text

```
ADF
↓
Azure IR
↓
Azure SQL
```

and:

text

```
ADF
↓
Self-hosted IR
↓
On-Prem SQL Server
```

Connection:

Linked Service tells ADF **what connection**; IR determines **how/where data movement executes**.

PHASE 2 – DATA MOVEMENT

Day 5 – Datasets

Cover:

- What is Dataset?
- Dataset vs Linked Service
- File dataset
- Table dataset
- Parameterized datasets
- Dataset location/schema

Simple distinction:

text

Linked Service
= Connection

Dataset
= Data being accessed

Day 6 – Pipelines & Activities

Cover:

- Pipeline
- Activity
- Sequential execution
- Parallel execution
- Activity dependencies
- Success / Failure / Completion paths

Example:

text

```
Lookup
  ↓
Copy
  ↓
Stored Procedure
  ↓
Notification
```

Day 7 – Copy Activity

Deeply cover the most important ADF activity.

Cover:

- Source
- Sink
- Source query
- Copy settings

- Mapping
- File movement
- SQL → ADLS
- ADLS → SQL
- Copy performance basics
- Parallel copy

Example:

text

```
SQL Server
  ↓
Copy Activity
  ↓
ADLS Gen2
```

Day 8 – Mapping Data Flow

Cover:

- What is Data Flow?
- Mapping Data Flow
- Source
- Transformation
- Sink
- Data Flow vs Copy Activity
- When to use Data Flow

Example:

text

```
Source
  ↓
Filter
  ↓
Join
  ↓
Derived Column
  ↓
Aggregate
  ↓
```

Important distinction:

Copy Activity moves data.

Data Flow transforms data visually.

PHASE 3 – CONTROL FLOW

Day 9 – Lookup + Get Metadata + ForEach

These three work together very often.

Cover:

Lookup

Read configuration/control data.

Get Metadata

Inspect files/folders.

ForEach

Repeat processing for multiple items.

Connected example:

text

```
Metadata Table
  ↓
Lookup
  ↓
ForEach
  ↓
Copy each table
```

This prepares the viewer for reusable pipelines.

Day 10 – Parameters vs Variables

Cover:

- Pipeline parameters
- Dataset parameters
- Activity parameters

- Variables
- Set Variable
- Append Variable
- Parameter vs Variable

Simple rule:

text

Parameter

= Input

Variable

= Runtime value

Day 11 – Expressions & Dynamic Content

Cover:

- Dynamic content
- Expressions
- String functions
- Date functions
- Logical functions
- Pipeline parameters
- Dataset parameters
- Referencing variables
- Building dynamic paths

Example:

text

/raw/2026/10/02/orders/

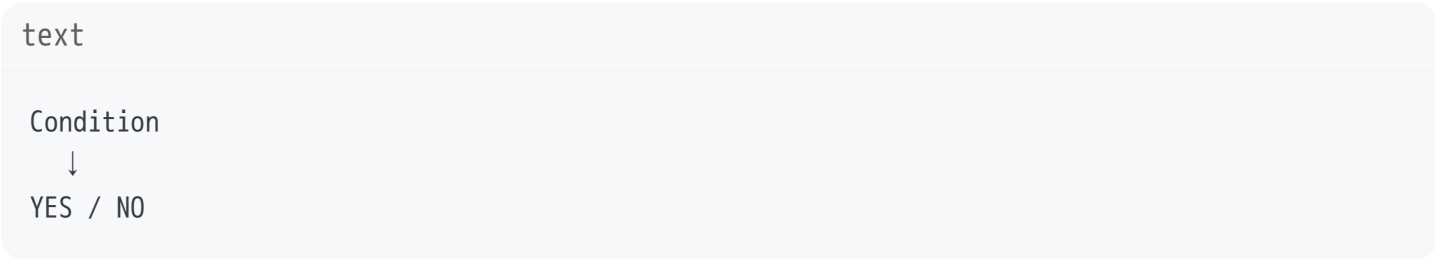
generated dynamically.

This is one of the most important ADF skills.

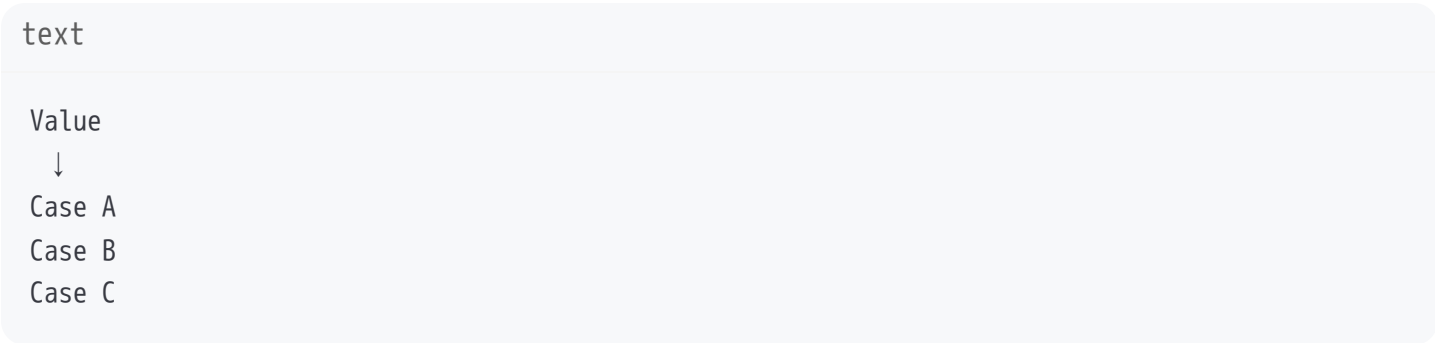
Day 12 – If Condition + Switch + Until

Cover:

If Condition



Switch



Until

Repeat until a condition becomes true.

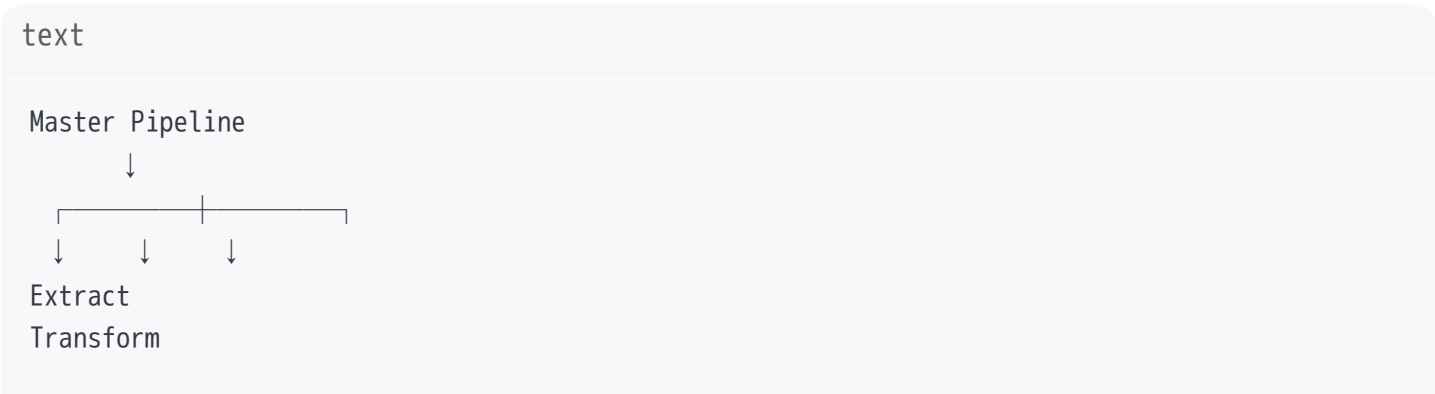
Use real examples rather than definitions.

Day 13 – Execute Pipeline + Control Flow Patterns

Cover:

- Execute Pipeline
- Parent pipeline
- Child pipeline
- Reusable pipelines
- Dependency chains
- Modular pipeline architecture

Example:



This introduces proper pipeline architecture.

PHASE 4 – TRIGGERS & INCREMENTAL LOADING

Day 14 – Debug vs Trigger + Triggers

Cover:

- Debug
- Trigger Now
- Schedule Trigger
- Tumbling Window Trigger
- Event Trigger
- Trigger parameters
- Dependency between trigger and pipeline

Explain the important difference:

Debug is for development/testing.

Trigger is for actual execution scheduling.

Day 15 – Full Load vs Incremental Load in ADF

Now bring your Core DE concepts into ADF implementation.

Cover:

- Full load pipeline
- Incremental load pipeline
- Source query
- Last modified column
- Watermark table
- Lookup watermark
- Copy changed records
- Update watermark

Architecture:

text

```
Control Table
↓
Lookup Watermark
↓
Source Query
↓
Copy Changed Data
↓
Update Watermark
```

Day 16 – Metadata-Driven ADF Pipeline

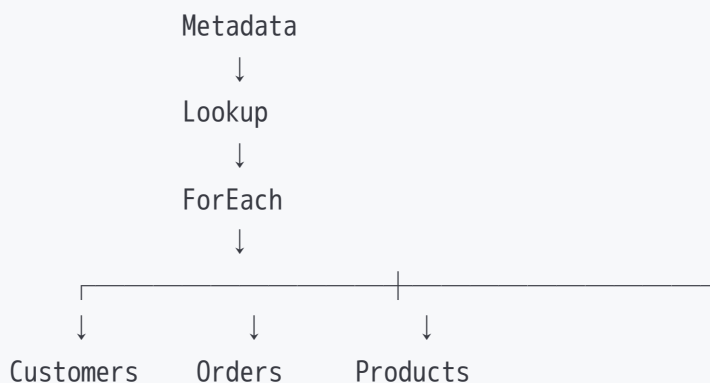
This should be one of the strongest videos.

Cover:

- Control table
- Metadata
- Lookup
- ForEach
- Parameters
- Dynamic datasets
- Dynamic source/sink
- Full vs incremental configuration

Example:

text



This is where the viewer moves from **basic ADF user** → **reusable pipeline builder**.

PHASE 5 – REAL-WORLD ADF INTEGRATION

Day 17 – Web Activity + REST API

Cover:

- REST API basics
- Web Activity
- GET / POST
- Headers
- Authentication
- API response
- Passing dynamic values
- Calling an API from ADF
- Using response in downstream activity

Example:

text

```
ADF
↓
Web Activity
↓
REST API
↓
JSON Response
↓
Pipeline
```

Day 18 – Error Handling + Retry + Failure Recovery

Cover:

- Success dependency
- Failure dependency
- Completion dependency
- Retry
- Timeout
- Error messages
- Failure notification
- Logging

- Rerun strategy
- Idempotent pipeline design

Example:

text

```
Copy
↓
FAILED
↓
Retry
↓
Success
```

Then:

text

```
Failure
↓
Log
↓
Alert
↓
Engineer
```

PHASE 6 – PRODUCTION ADF

Day 19 – ADF Security + Git + CI/CD + Monitoring

This day brings ADF into real project usage.

Cover:

Security

- Managed Identity
- Azure Key Vault
- Secrets
- RBAC
- Secure connections

Development

- Git integration

- Branches
- Collaboration

Deployment

text

Dev
↓
Test
↓
Prod

Monitoring

- Pipeline runs
- Activity runs
- Trigger runs
- Alerts
- Monitoring dashboard

Day 20 — Build a Complete Production-Style ADF Pipeline

This is the final payoff.

Build one complete scenario:

SQL Server → ADLS → transformation → target

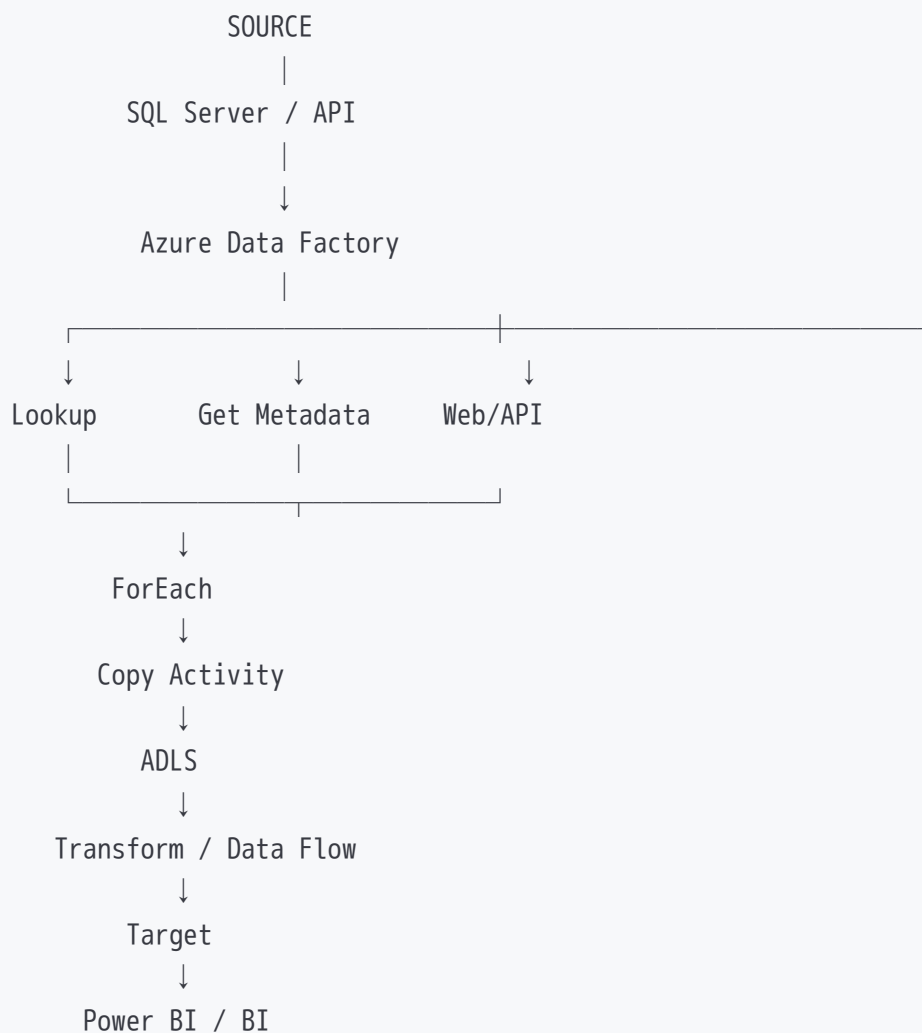
with:

- Linked Service
- Dataset
- Integration Runtime
- Pipeline
- Copy Activity
- Lookup
- Parameters
- Variables
- Dynamic Content
- ForEach

- Metadata-driven design
- Incremental load
- Watermark
- Error handling
- Retry
- Monitoring
- Trigger

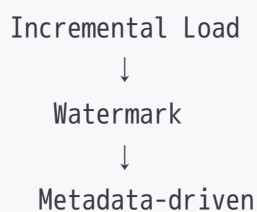
Architecture:

text



Supporting logic:

text



↓
Idempotent
↓
Error Handling
↓
Monitoring

Final 20-Day ADF Roadmap

Day	Topic
01	What is Azure Data Factory?
02	ADF Main Components
03	Linked Services
04	Integration Runtime
05	Datasets
06	Pipelines & Activities
07	Copy Activity
08	Mapping Data Flow
09	Lookup + Get Metadata + ForEach
10	Parameters vs Variables
11	Expressions & Dynamic Content
12	If Condition + Switch + Until
13	Execute Pipeline + Modular Pipelines
14	Debug vs Trigger + Triggers
15	Full Load vs Incremental Load in ADF
16	Metadata-Driven Pipelines
17	Web Activity + REST APIs
18	Error Handling + Retry + Recovery
19	Security + Git + CI/CD + Monitoring
20	Complete Production ADF Pipeline

The learning connection

text

```
ADF Basics
↓
Components
↓
Connections
↓
Data Movement
↓
Control Flow
↓
Dynamic Pipelines
↓
Incremental Processing
↓
Metadata-Driven Design
↓
APIs
↓
Error Handling
↓
Security + CI/CD + Monitoring
↓
REAL PROJECT
```

This keeps the **Core DE 20 days** separate from the **ADF 20 days**. After ADF, your next series can be **Spark + PySpark**, starting from Spark architecture and DataFrames and then moving into transformations, joins, partitions, shuffle, optimization and real PySpark pipelines.

2026-10-02 17:34:53 You 

good more think and proper almost 1 min reels like content and proper cover as past like script with good one cover ok . give day 1 adf

 ChatGPT gpt-5-6-t-mini

Day 1 — What is Azure Data Factory?

“Data Engineering mein ek common problem hai...

data ek jagah nahi hota.

Maan lo ek company ka customer data SQL Server mein hai...

orders CSV files mein hain...

aur kuch data kisi REST API se aa raha hai.

Ab in sab data sources ko ek central data platform tak regularly kaise laoge?

Manually?

Obviously nahi.

Yahin pe aata hai...

Azure Data Factory – ADF.

ADF ko simple language mein samjho...

ye ek **cloud-based data integration and orchestration service** hai.

Matlab ADF different data sources se data ko connect kar sakta hai...

move kar sakta hai...

aur poore data workflow ko **schedule aur orchestrate** kar sakta hai.

For example...

text



Suppose mujhe SQL Server se daily orders uthakar

Azure Data Lake mein load karne hain.

ADF pipeline ye poora workflow automate kar sakti hai.

Aur sirf ek baar run karna nahi...

aap ise schedule kar sakte ho...

dependencies laga sakte ho...

failure handle kar sakte ho...

aur monitor bhi kar sakte ho.

Ab ek important point...

ADF khud har type ka data processing engine nahi hai.

Iska main strength hai...

connect, move, and orchestrate.

Heavy transformation ke liye

aap Databricks, Azure Synapse, SQL, Fabric jaise services ko ADF workflow ke andar trigger kar sakte ho.

So simple way mein yaad rakho...

ADF is the control center of your data pipeline.

It connects your sources...

moves the data...

and coordinates what happens next.

Aur ab jab humein pata hai **ADF karta kya hai...**

next question hai...

ADF ke andar actually kaun-kaun se components milke ye pipeline banate hain?

Chalo next samajhte hain...

ADF ke Main Components.

2026-10-02 17:49:38 You 

continue next

 ChatGPT gpt-5-6-t-mini

Day 2 – ADF Main Components

“Kal humne samjha tha...

Azure Data Factory actually karta kya hai.

But ab jab aap ADF open karoge...

toh sabse pehle confusion yahi hota hai...

ye Pipeline, Dataset, Linked Service, Activity... ye sab kya hai?

Chalo ek simple example se samajhte hain.

Suppose mujhe SQL Server se
customer data uthakar
Azure Data Lake mein load karna hai.

Sabse pehle mujhe workflow define karna hoga...

and that workflow is called a **Pipeline**.

Pipeline basically batati hai...

kaam kis sequence mein hona hai.

Pipeline ke andar jo individual kaam hote hain...

unhe bolte hain **Activities**.

For example...

data copy karna...

Lookup karna...

ya kisi aur process ko trigger karna.

Ab ADF ko SQL Server se connect kaise karna hai?

Us connection ke liye use hota hai...

Linked Service.

Linked Service basically batata hai...

connection kahan hai, aur usse connect kaise karna hai.

But sirf connection enough nahi hai.

ADF ko ye bhi pata hona chahiye...

actual data kya hai?

Table kaunsi hai...

file kaunsi hai...

path kya hai...

Ye information represent karta hai...

Dataset.

So agar simple flow dekho...

text

```
Pipeline
  ↓
Activity
  ↓
Dataset
  ↓
Linked Service
  ↓
Source / Target
```

Aur ek aur important component hai...

Integration Runtime.

Ye woh compute infrastructure hai
jo ADF ko data movement aur connectivity ke liye required environment provide karta hai.

Toh simple language mein...

Pipeline = workflow

Activity = actual task

Linked Service = connection

Dataset = data reference

Integration Runtime = connectivity / execution infrastructure

Bas in components ko samajh lo...

ADF ka structure kaafi clear ho jayega.

Aur ab next important question...

Linked Service exactly kaise connection banata hai?

Aur SQL Server, ADLS ya kisi API se connect karte waqt
ADF authentication kaise handle karta hai?

Yahin se samajhte hain...

Linked Services."

2026-10-02 22:46:32 You 

continue in proper

Day 3 – Linked Services in Azure Data Factory

“Abhi humne ADF ke main components samjhe...

Pipeline kya hai...

Activity kya hai...

Dataset kya hai...

But ab ek simple question...

ADF ko pata kaise chalega ki data actually kahan hai?

Suppose mujhe SQL Server se data read karna hai.

ADF ko SQL Server ka address chahiye...

database chahiye...

authentication chahiye...

Basically...

connection information chahiye.

Aur ADF mein isi kaam ke liye use hota hai...

Linked Service.

Simple language mein...

Linked Service is the connection information for an external service.

For example...

agar source SQL Server hai...

text

```
graph TD
  ADF[ADF] --> LS[Linked Service]
  LS --> SS[SQL Server]
```

Aur agar destination Azure Data Lake hai...

text

```
graph TD
  ADF[ADF] --> LS[Linked Service]
  LS --> ADL[Azure Data Lake]
```

Matlab ADF mein SQL Server ke liye ek connection define kar sakte ho...

aur ADLS ke liye doosra connection.

Ab yahan ek common confusion hota hai...

Linked Service aur Dataset same nahi hain.

Linked Service basically bolta hai...

“Main kis service se connect kar raha hoon?”

Dataset bolta hai...

“Us service ke andar mujhe kaunsa data access karna hai?”

For example...

SQL Server ek Linked Service hai...

aur uske andar **Customers table** ko represent karne wala Dataset hai.

So think like this...

Linked Service = building ka address.

Dataset = building ke andar ka specific room.

Aur security bhi yahin important ho jaati hai.

ADF ko credentials hardcode karne ke bajay...

production environments mein

Managed Identity, Key Vault jaise secure authentication methods use kiye ja sakte hain.

So ek simple rule yaad rakho...

Linked Service tells ADF how to connect.

Dataset tells ADF what data to work with.

Aur ab ek aur question...

ADF ko connection mil gaya...

but ye data movement actually execute kahan karega?

Yahin se samajhna important hai...

Integration Runtime.”

continue next topic and proper good one



Day 4 — Integration Runtime (IR) in Azure Data Factory

“Abhi tak humne dekha...

Pipeline kya hai,
Activity kya hai,
Linked Service kya hai.

But ek important question...

ADF actually source se data connect aur move karega kahan se?

Yahin pe aata hai...

Integration Runtime — IR.

Simple language mein samjho...

Integration Runtime ADF ka compute aur connectivity infrastructure hai,

jo supported data integration activities ko execute karne aur data sources tak connect karne mein help karta hai.

Ab maan lo...

Aapka SQL Server **Azure mein hai,**
aur data ko ADLS mein copy karna hai.

Toh **Azure Integration Runtime** use kar sakte ho.

But suppose...

SQL Server aapke **company ke on-premise network** mein hai.

Internet se directly accessible nahi hai.

Toh yahan use karenge...

Self-hosted Integration Runtime.

Ye aapke network ke andar machine par install hota hai
aur ADF ko private data source tak securely connect karne mein help karta hai.

Aur ek aur option hai...

Managed VNet Integration Runtime,

jahan ADF managed virtual network ke through connectivity provide kar sakta hai.

Toh simple rule yaad rakho...

Cloud data source → Azure IR

Private / On-premise data → Self-hosted IR

Aur ek important confusion...

Linked Service aur IR same nahi hain.

Linked Service batata hai —

kisse aur kaise connect karna hai.

IR provide karta hai —

connection aur data integration activity ke liye required environment.

Toh ab connection clear hai...

But actual data ko **copy** kaise karenge?

Uske liye aata hai...

Dataset.”